Artificial Intelligence

automation like
↳ scheduling, eg. airline routing
route planning, eg. google maps

environment,
is everything in world which surrounds the agent, but it is not part of an agent itself.

spam classifiers
Automated help desks
Fraud Detection
Web search engines

(surrounding environs input → output)

If an agent sensor can sense or access the complete state of an environment at each point of time then it is a fully observable environment, else it is partially observable.

eg. Tic-Tac-Toe, chess (1 player gets to see the whole board)

here, a fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world.

An agent with no sensors in all environments then such an environment is Unobservable.

eg. automated taxi driving.

poker (a player gets to see only his own cards, not the cards of everyone in game)

*single agent operating → single agent environment.

*multiple agent operating → multi-agent environ.

*The agent design problems in the multi-agent environment are different from single agent envir

eg. Single Agent (crossword Puzzle);

• Multi Agent (chess)

*If an agent's current state & selected action can completely determine the next state of environment → deterministic environment.

eg. Tic-Tac-Toe; chess

* If stochastic environment is random in nature and cannot be determined completely by an agent. (random ✓) eg. Poker.

* In a deterministic, fully observable environ, agent does not need to worry about uncertainty. (random ✗)

eg. eg. 8 puzzle game.

* Episodic environment, series of one-shot actions, only current percept is required for action.

eg. Part picking Robot, Vacuum cleaner. Image Recognition.

* Sequential environment, agent requires memory of past actions to determine the next best actions. (depends on previous state)

eg. Chess, Taxi driving.

* Static environment, easy to ded, an agent doesn't need to continue looking at world while deciding for an action.

eg. Crossword Puzzle.

* Dynamic environment,
agents need to keep looking at world at each action. (changeable)

eg. Taxi Driving

* Discrete environment,
finite number of percepts and actions that can be performed within it, then such an environment,

eg. Crossword Puzzle, Poker.

* eg. of Continuous environment, Taxi Driving
(infinite steps)

* Known and Unknown are not actually a feature of environment, but it is an agents state of knowledge to perform an action.

* Results for all actions are known to agent, Known environment.

* Unknown environment, agent needs to learn how it works in order to perform an action.

* known → partially observable.

unknown → fully observable.

Characteristics of environments

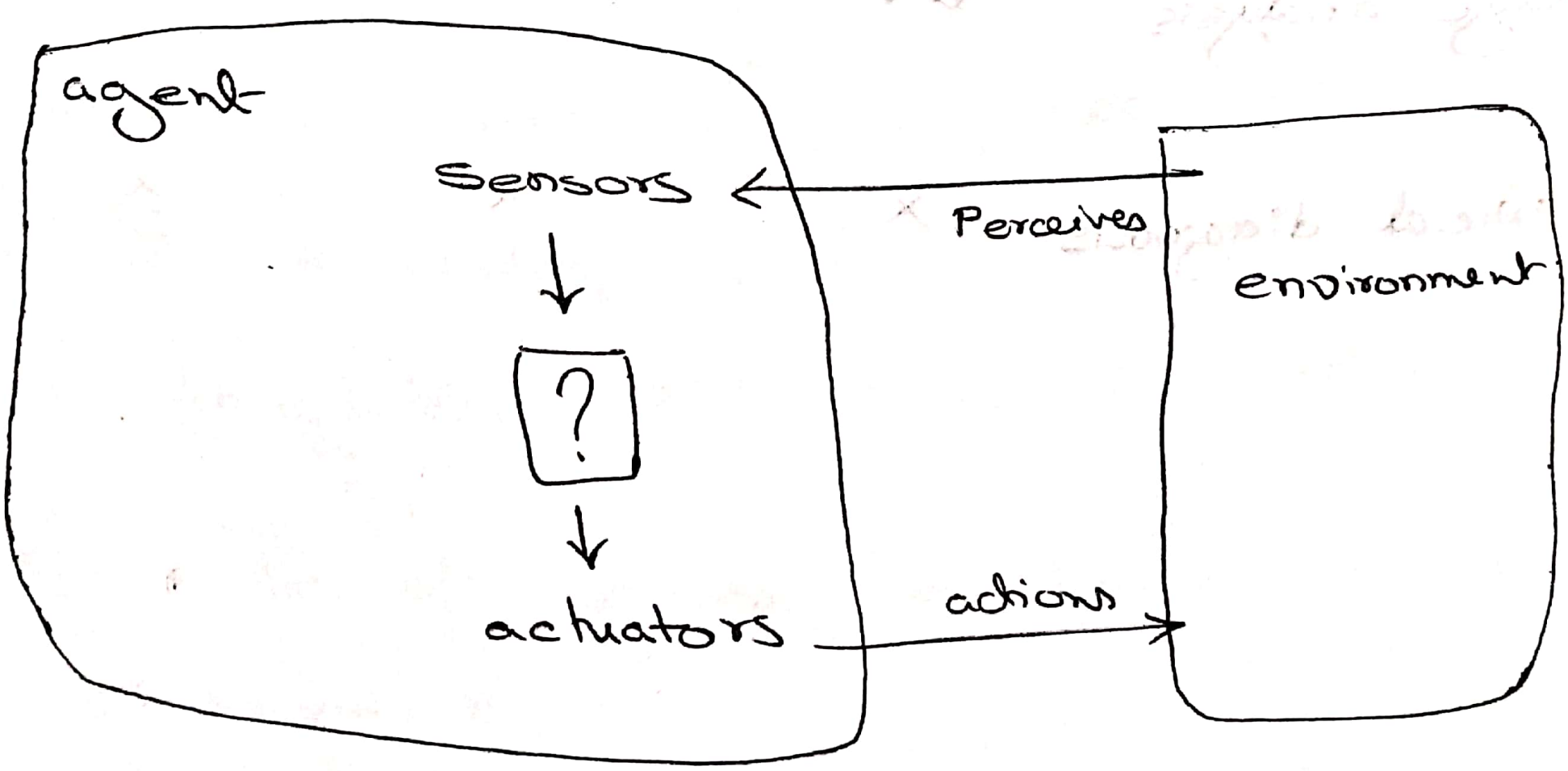
	Fully observable (complete knowledge of envi.	Deterministic (action & current state determine next level)	Episodic (next action sprevious actin)	Static	Discrete (finite)	Single Agent
crossword	✓	✓	×	✓	✓	✓
poker	×	×	×	✓	✓	×
Taxi driving	×	×	sequential ×	dynamic ×	continu ×	×
Image analysis	✓	✓	✓	Semi	×	✓
Medical diagnosis	×	×	×	×	×	✓

Structure of agents.

An intelligent agent perceives its environment via sensors & acts rationally upon that environ. with its actuators.

* A discrete agent receives percepts one at a time, and maps this percept sequence to a sequence of discrete actions.

properties
- ↳ autonomous
- ↳ reactive to environment.
- ↳ pro-active (goal-directed)
- ↳ interacts with other agents via environ.



Robotic Agent / Software Agent ; Human Agent

actuator : pink wheel.

actuator: PC

sensors: keyboard

Incase of Humans,

Sensors: eyes (vision), ears (hearing), skin (touch), tongue (gustation), nose (olfaction), neuromuscular system (proprioception).

Percepts:

at lowest level - electrical signals from these sensors.

after preprocessing - objects in the visual field (location, textures, colors ...)

auditory systems (pitch, loudness, direction ...)

actuators: limbs, digits, eyes, tongue.

actions: lift a finger, turn left, run, walk, carry an object.

Smashers

Types of Agents,

① Simple Reflex Agents,

based on condition-action rules, implemented with an appropriate production system. They are stateless devices wh do not have memory of past world states.

② Agents with memory (model/knowledge),

have internal state, which is used to keep track of past states of world.

③ Agents with goals,

in addition to state information, have goal information that describes desirabl situations.

take future events into consideration.

④ Utility-based Agents,

base their decisions on classic axiomatic ~~utility~~ utility theory in order to act rationally.

⑤ Table-driven agents,

use a percept sequence/action table in memory to find the next action.

implemented by a (large) lookup table.

Table-driven agents, (1-1) simplex

Table lookup of percept-action pairs mapping from every possible perceived state of to the optimal action for that state.

probs // energy consumption (↑)

- Too big to generate & to store (chess has 10^{150} states)
- stuck in a loop.
- No agent could ever learn all the right table entries from its experience.
- Not feasible for bigger prob,
- No guidance about how to fill in the table entries. - Not scalable with change environ.
- Not adaptive to changes in environment requires entire table to be updated if changes occur
- Looping - can't make actions conditioned on previous actions/states.

continuous → (condition)

if temp ≥ 25 turn on else turn off

1 Simple Reflex agents,

immediate / spontaneous action.

Do not check past record.

based on current state.

based on If-Then rule.

eg. Vacuum cleaner.

probs

-Too big table to generate & to store.

-infinite loop.

- Do not work for partially observable.

- Do not involve multiple condition.

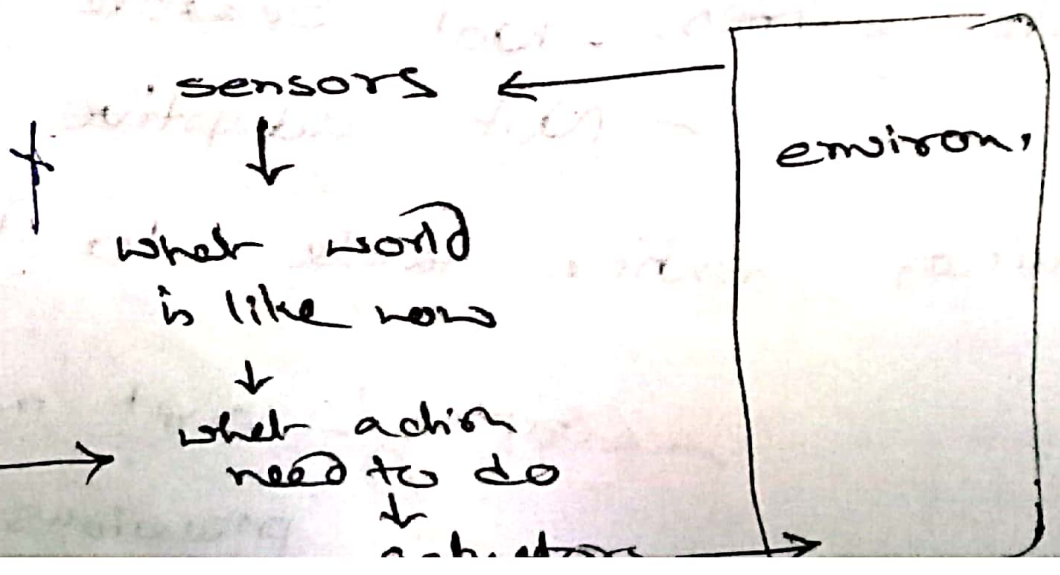
-No knowledge of non-perceptual parts of state.

-Not adaptive to environment change; requires collection of rules to be updated if changes occur.

- Can't make actions conditional on previous state.

solve give memory

A clean
B clean



function,

Percept sequence	action	depends on current ✓
[A, clean]	Right	
[A, Dirty]	Suck	
[A, clean] [A, dirty]	Suck	

2] Agents with Memory (model-based Agents)

knowledge based

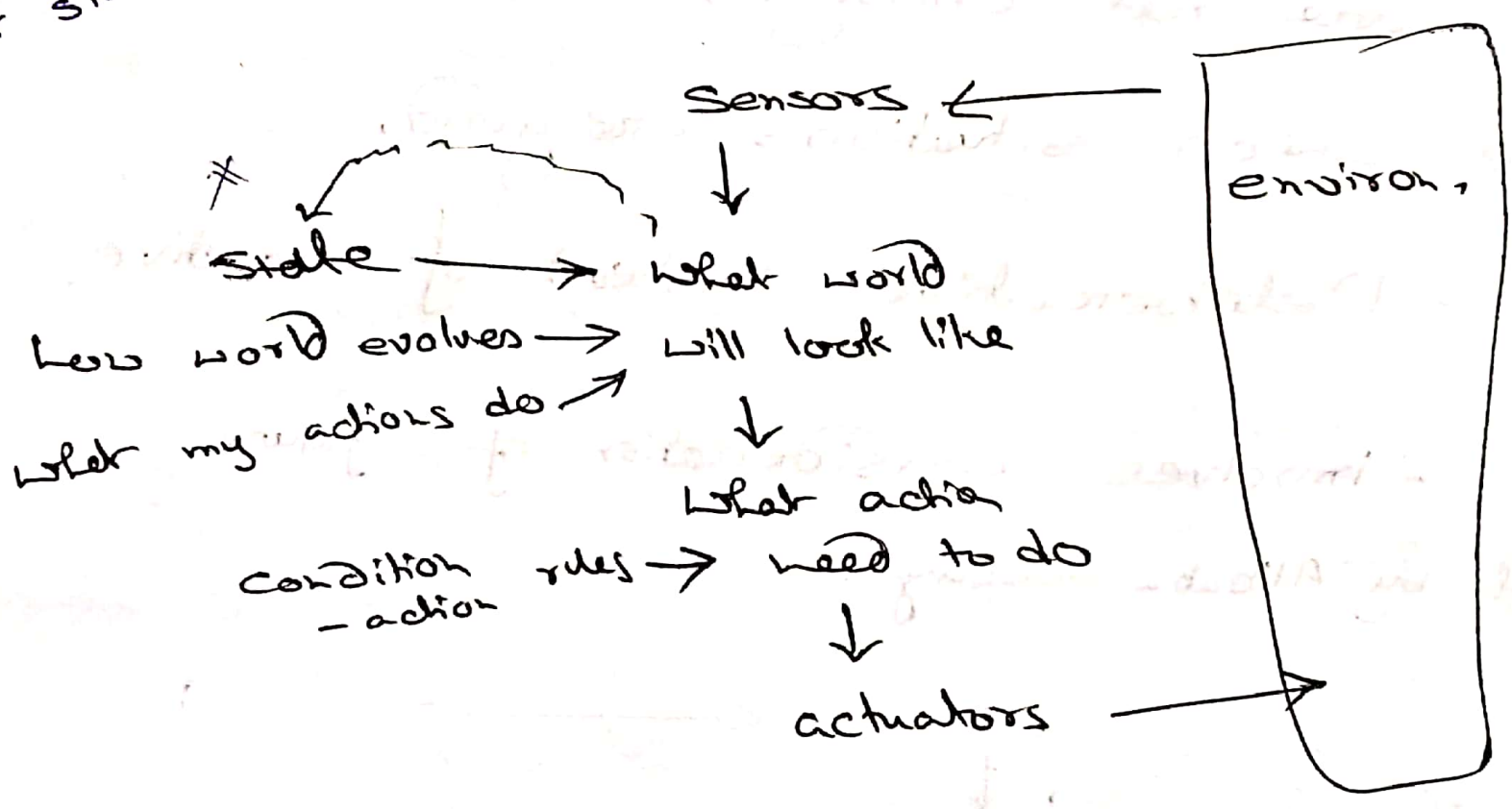
Store percept history

partially observable

adaptive ability to environ. change.

percept environ's
idea ↳ memory

*not stuck in loop.



eg. automated taxi driving system.

percepts: Video, sonar, speedometer, odometer, engine sensors, keyboard input, GPS

actions: steer, brake, horn, speak/display, accelerate.

goals: maintain safety, reach destination, obey laws, provide passenger comfort,

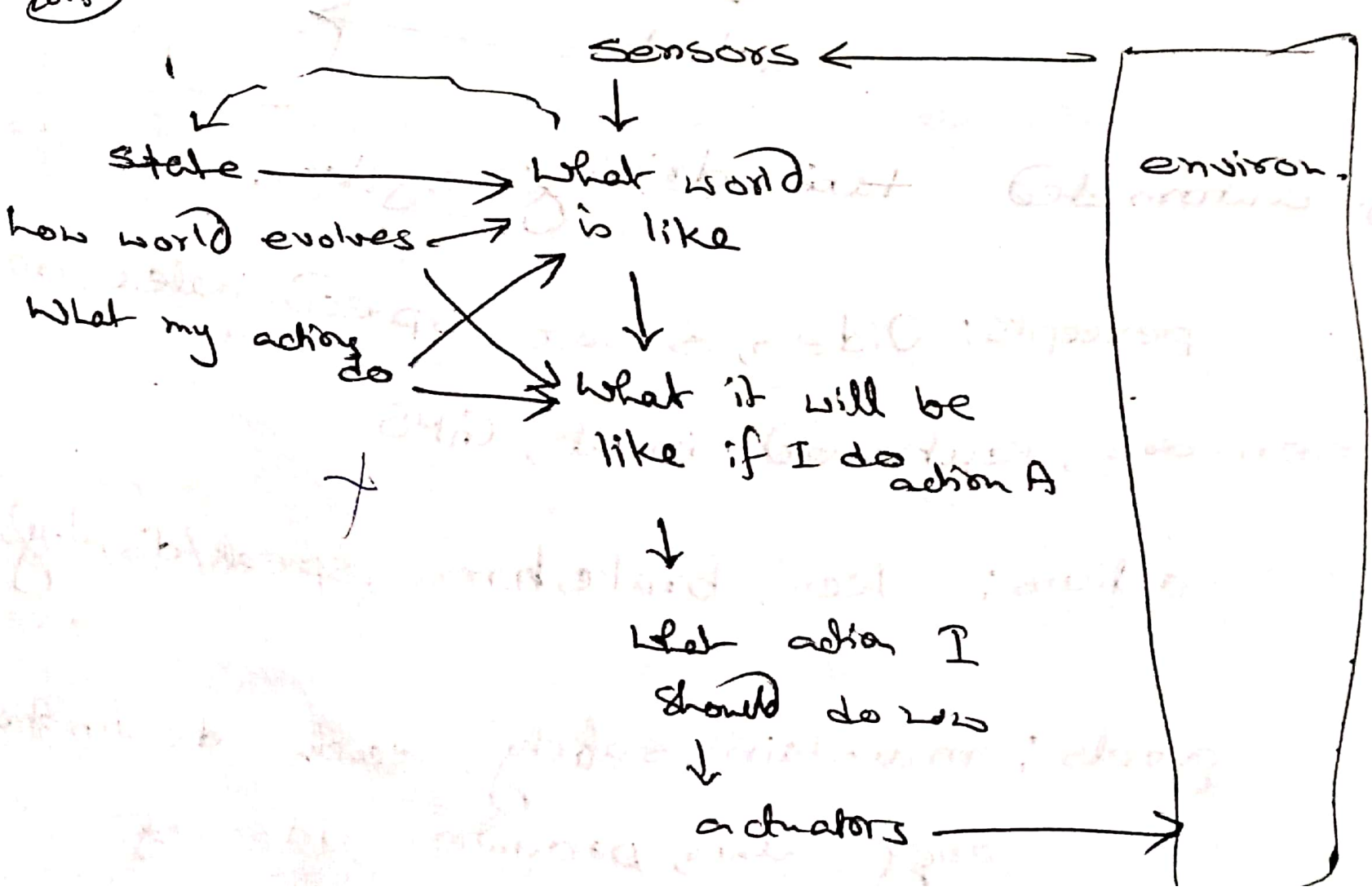
environment: freeways, traffic, weather, customers, urban streets

(most rational)

[3] Agents with goals (Goal-based)

- expansion of model based agent.
- choose actions so as to achieve a desired goal.
- Searching & Planning.
- Keep current state - track & past state are not enough, need to add goals to decide which situations are good.
- Deliberative instead of reactive.
- involves consideration of future.

eg. G+ Robot by Alibaba company. (2018)



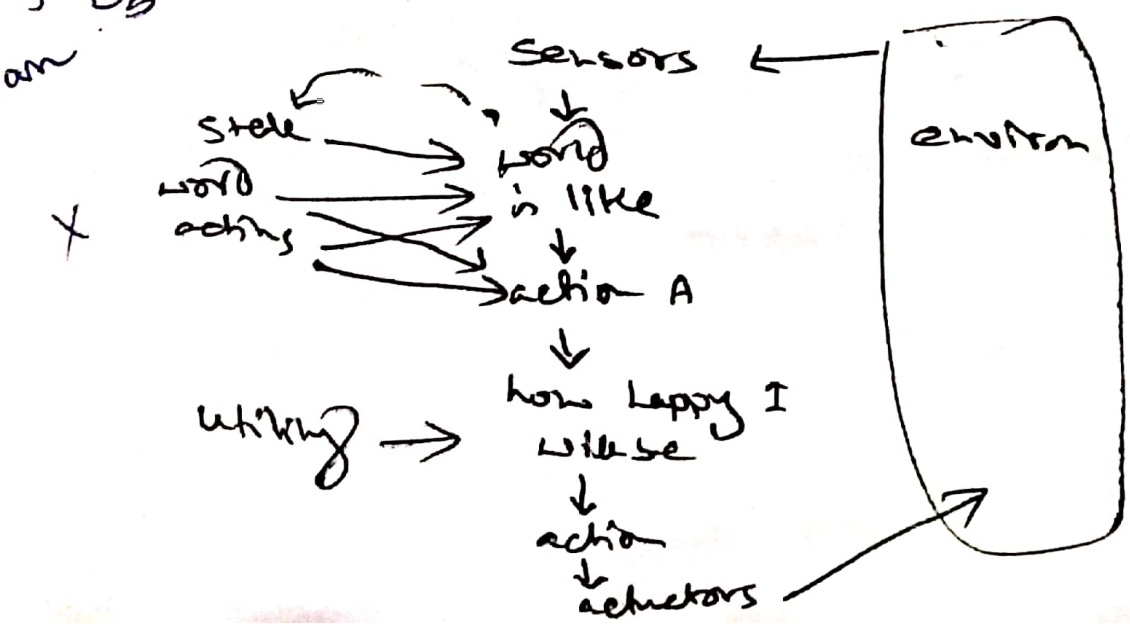
(considers turn) (decides (primary goal) (current world state, happy or not)

4] Utility-based agents,

- Focuses on utility or degree of happiness.
- When there are multiple possible alternatives, how to decide which one is best?
- Utility function $U: State \rightarrow Reals$ indicating a measure of success or happiness when at a given state,
- allows decisions comparing choice between conflicting goals, and choice between likelihood of success & importance of goal (if achievement is uncertain)

eg. Mars Lander on the surface of mars with obstacle in its way; utility function → choose best path for best output,

self-driving agent
- Reduce safety
- pedestrian safety



Goals of AI

Represent and store knowledge

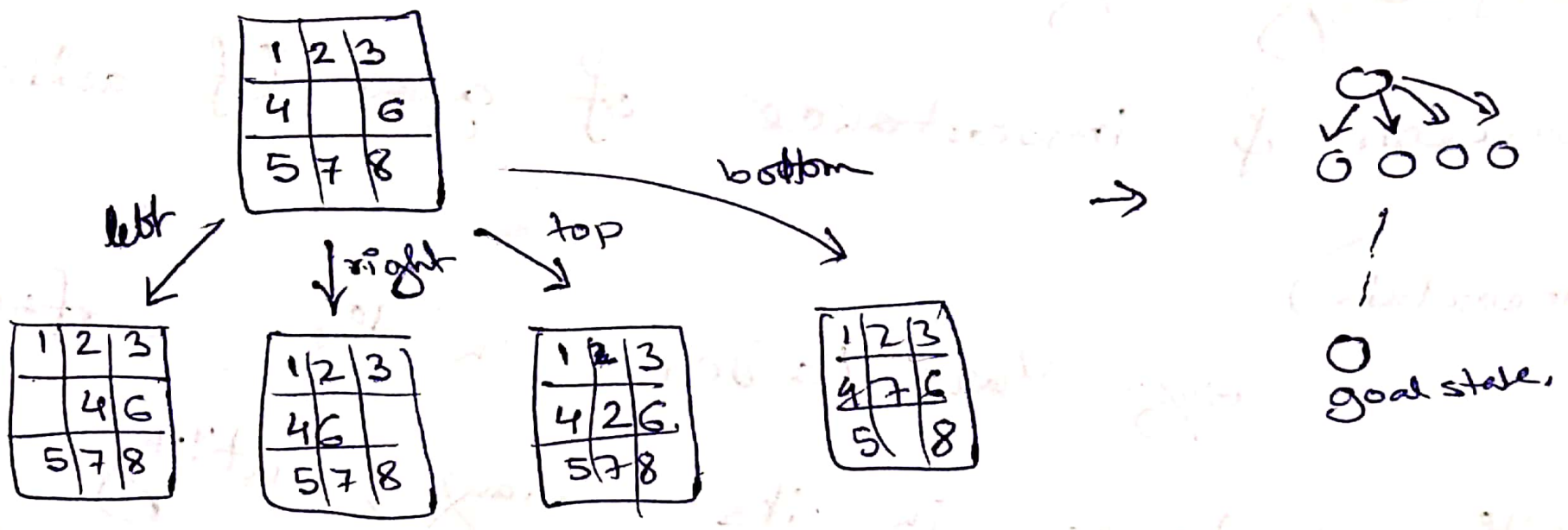
Retrieve and reason about knowledge

Behave intelligently in complex environ.

Develop interesting and useful applications.

Interact with people, agents and the enviro.

State_Space (graph)



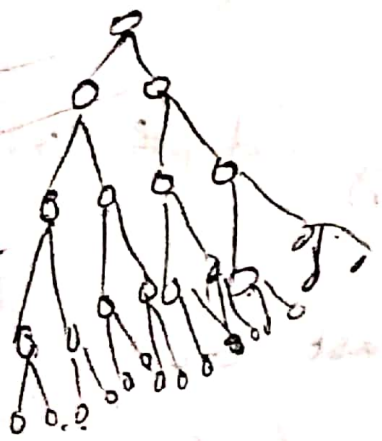
Uninformed Search Methods

↳ DFS
↳ BFS
↳ UCS (Uniform Cost Search)

x Real Life example,

Uninformed	Informed
Travelling without tourist guide.	Travelling with tourist guide

Search Strategies,



Uninformed (blind search)

- no path cost
- no steps cost.

It only know
↳ how to traverse tree

Informed (heuristic search)

calculate the path cost to find the optimise path from current to goal state.

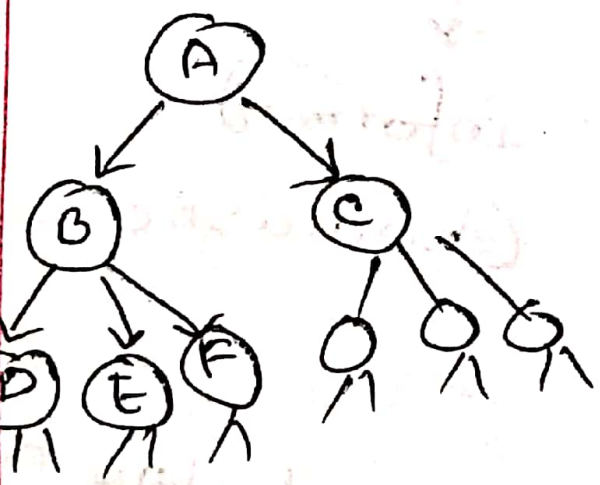
Uninformed Search

(Blind search)

① Search without infos

② No Domain Knowledge

③ Follow Brute Force Way.



Types of Uninformed,

① BFS : expand shallow node first.

② DFS : expand deepest node first.

③ DLS : DFS with limited depth.

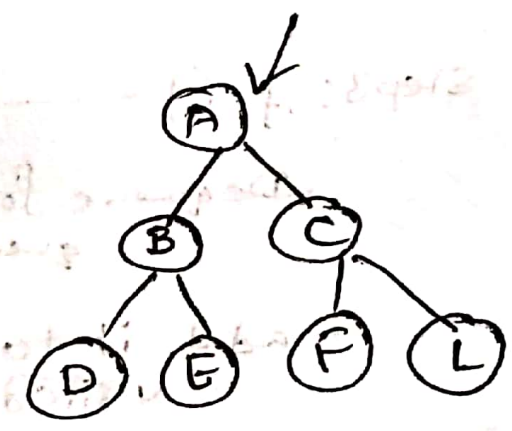
④ IDS : DLS With increasing depth limit

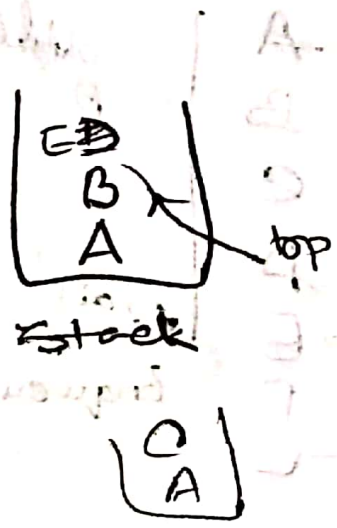
⑤ UCS : expand least cost node first.

Algo complexity (Space & Time)

DFS — Depth First Search Algo.

- Uninformed search < no domain knowledge / seach without information.
- Uses stack (LIFO)
- traverse / Search (depth wise)
- back track





Display: A B D E C F L

Algo

Step 1: Put Initial Node on Stack
Mark it as Visited
Display it

Step 2: if (node at top) has adjacent unvisited node.
- visit unvisited nodes
- mark as visited
- push it into Stack
- Display

else
pop the top element of Stack

Step 3: if (Stack empty)
Go to Step 2.

Time complexity: b^d

Space complexity: $b \times m^{(depth)}$

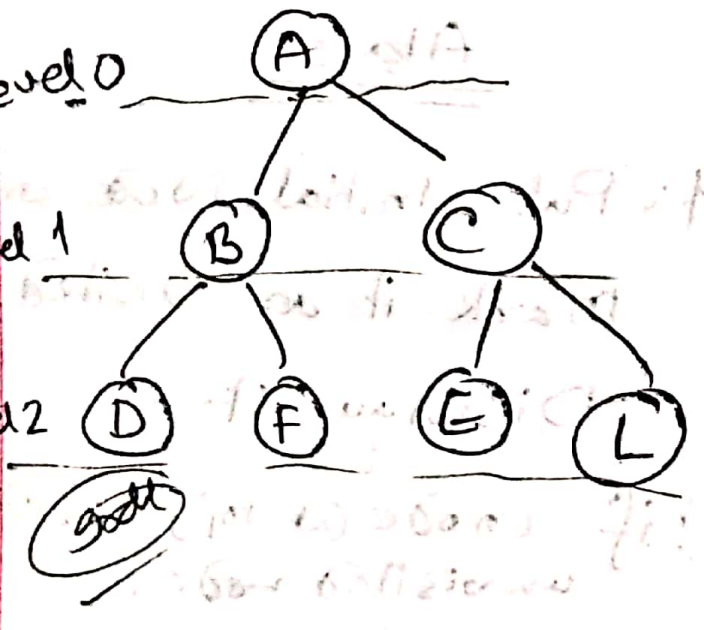
Complete: DFS not complete (infinite loop)

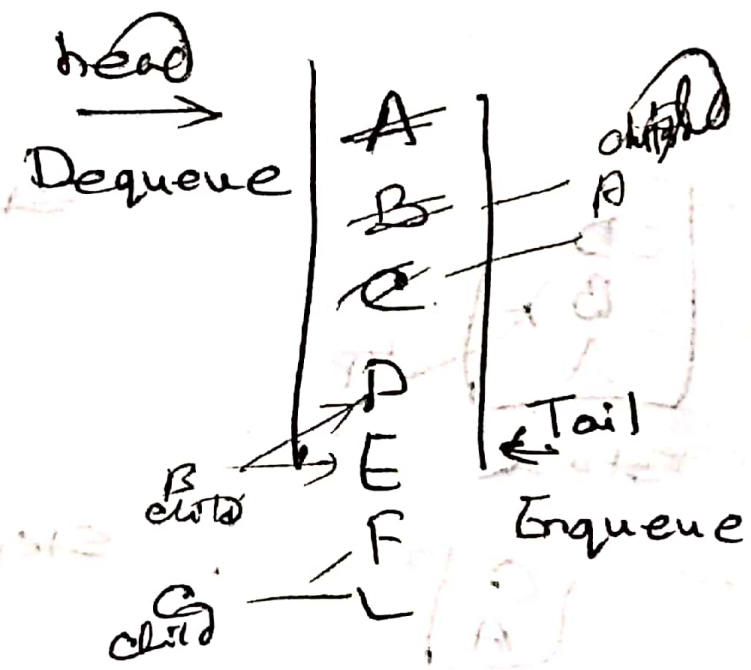
Optimal: No.

BFS

Breadth First Search Algo.

- Uninformed Search < no domain knowledge / search without info.
- uses Queue (FIFO)
- Traverse/Search by Level.





b = avg no. of children of a node

d = depth

2

2

visited node list

A B C

t.c b

s.c b

Complete ✓

optimal ✓

algo

Step1: Put initial node in queue.

Step2: if (1st node == goal node)

↳ go to step5

Step3: if (! — —)

- Dequeue from queue
- add it to visited
- enqueue all child

Step4: if (queue not empty)

→ go to step 2

Step5: exit.

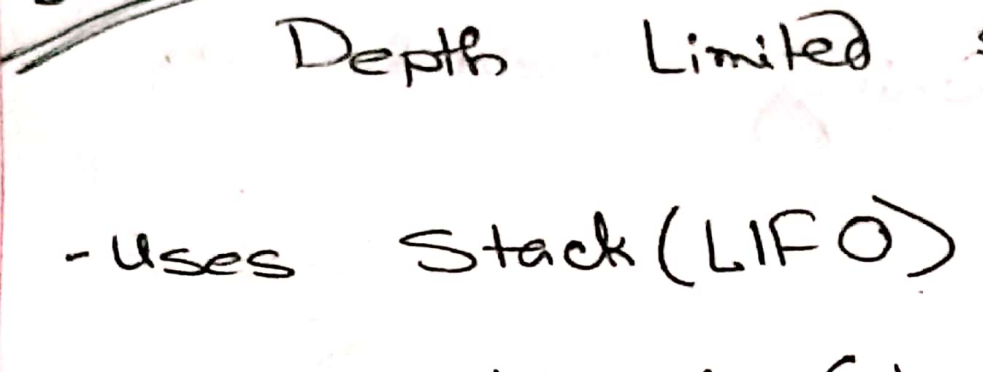
* more memory space,

DLS

Depth Limited Search Algo.

- Uses Stack (LIFO)
- traverse/search (depth wise)
- combination of (DFS + Binding Level)

DLS





T.C b^{Level}

S.C $b \times m^{(depth)}$

Complete No

Optimal No

algo

Step1: - Put initial node on stack
- Mark it as visited
- Display it,

Step2: if (node at top) has adjacent unvisited node,
- visit unvisited node
- mark it as visited
- Push it
- Display

else {
pop top element.

Step3: if (stack empty) go to step 2

IDS

Iterative Deepening Search algo

- Uses Slack,
- combination (BFS+DFS)
- Do DFS in a BFS fashion

General idea

→ Perform DLS for depth (L=0)

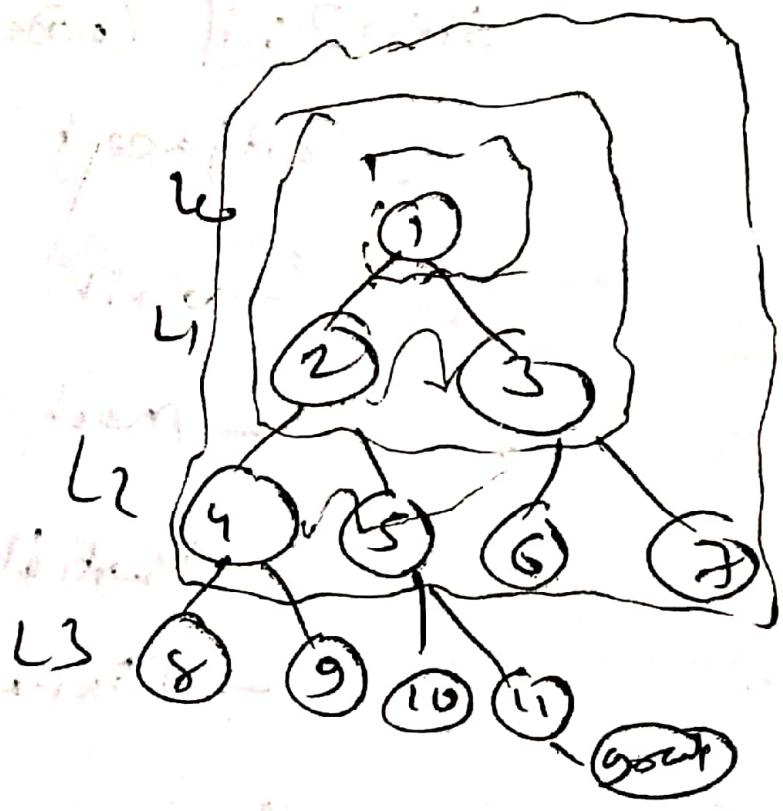
↳ if found goal, return it.

→ else perform for depth (L=1)

↳ if found goal, return it.

→ or else

↳ Repeat DLS for Depth ↳ Until found goal.



Depth	Iteration
0	1
1	1, 2, 3
2	1, 2, 4, 5, 3, 6, 7
3	1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 7

$T.C = b^{DL}$

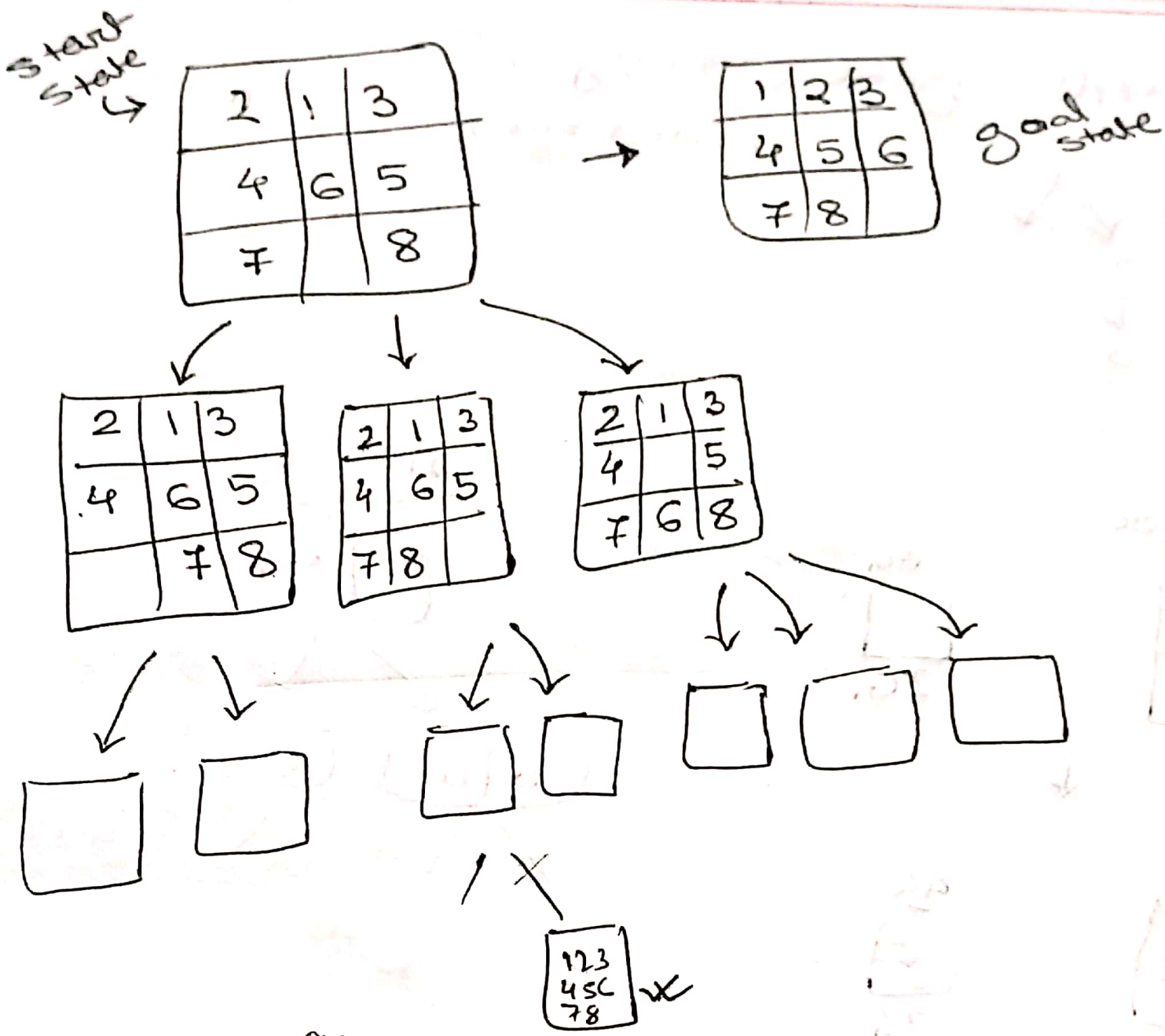
$S.C = b \times L$

Complete = Yes

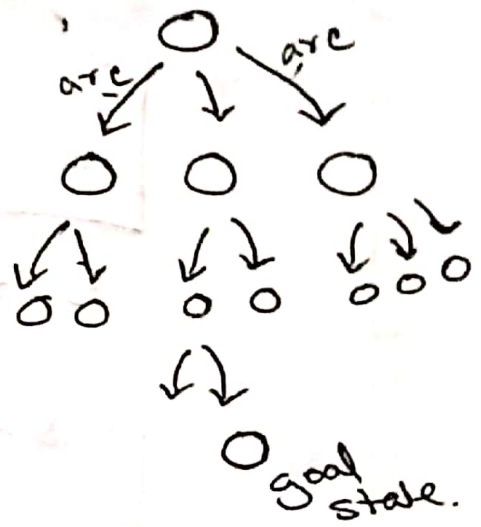
422

4th class

State space graph (all possible states)



** Write as tree



State State Stagl

terminology
↳ Node
↳ Arc
↳ start state (1) always.
↳ goal - { } set

tic-tac-toe
↳ 3^9 states

checkers ↳ 10^{40}

Rubics cube ↳ 10^{19} states

chess ↳ 10^{120} state

open visited

visited (stack) open

expand (node ber kaha)

closed (sb ses bhi)

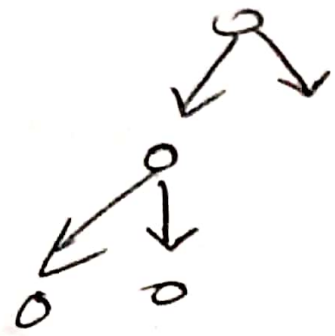
** searching algo's property

↳ completeness
↳ time complexity
↳ Space "
↳ optimality (admissibility)

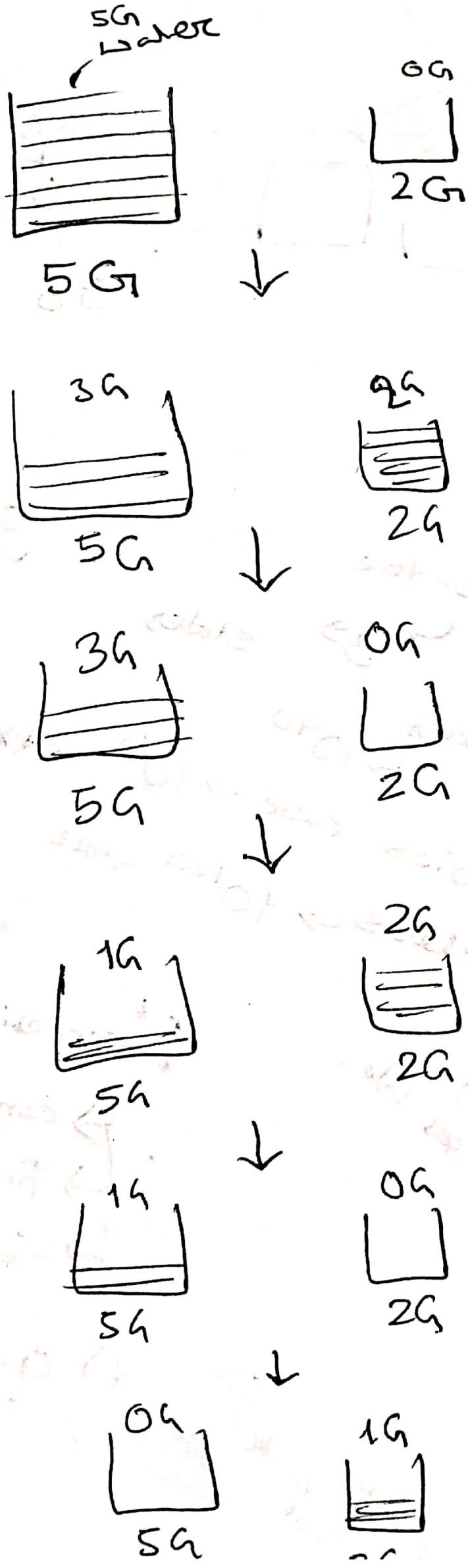
shortest path ? start goal

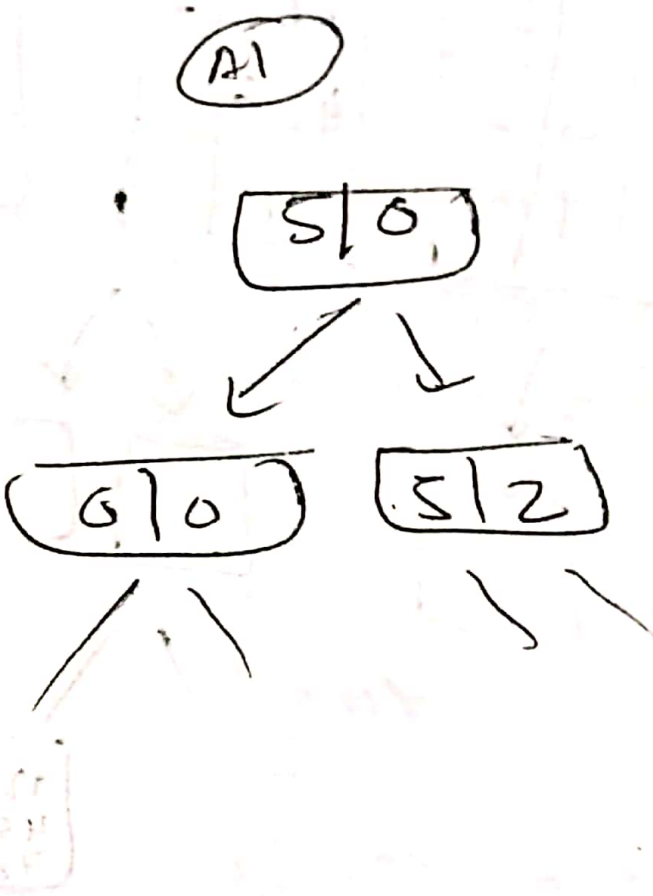
8 puzzle game (one player)

(control mine)



prob





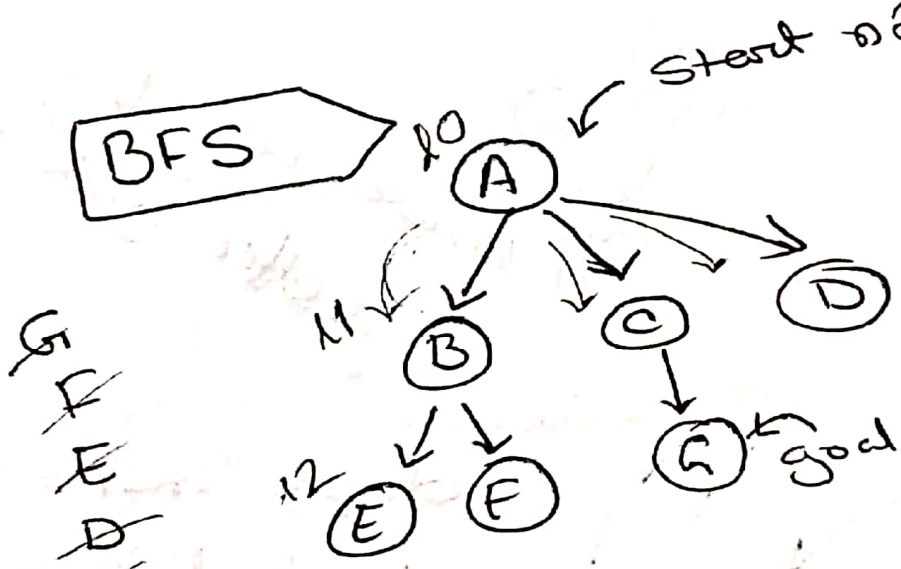


Uninformed search;
(limited State Space)
(expensive)

eg. bfs
dfs
↳ DLS (depth limited s—)
↳ IDS (Iterative deepening s—)
UCS (uniform cost search)
BDS (Bidirectional search)

② Informed
(Additional info;
✓good into present → heuristic)

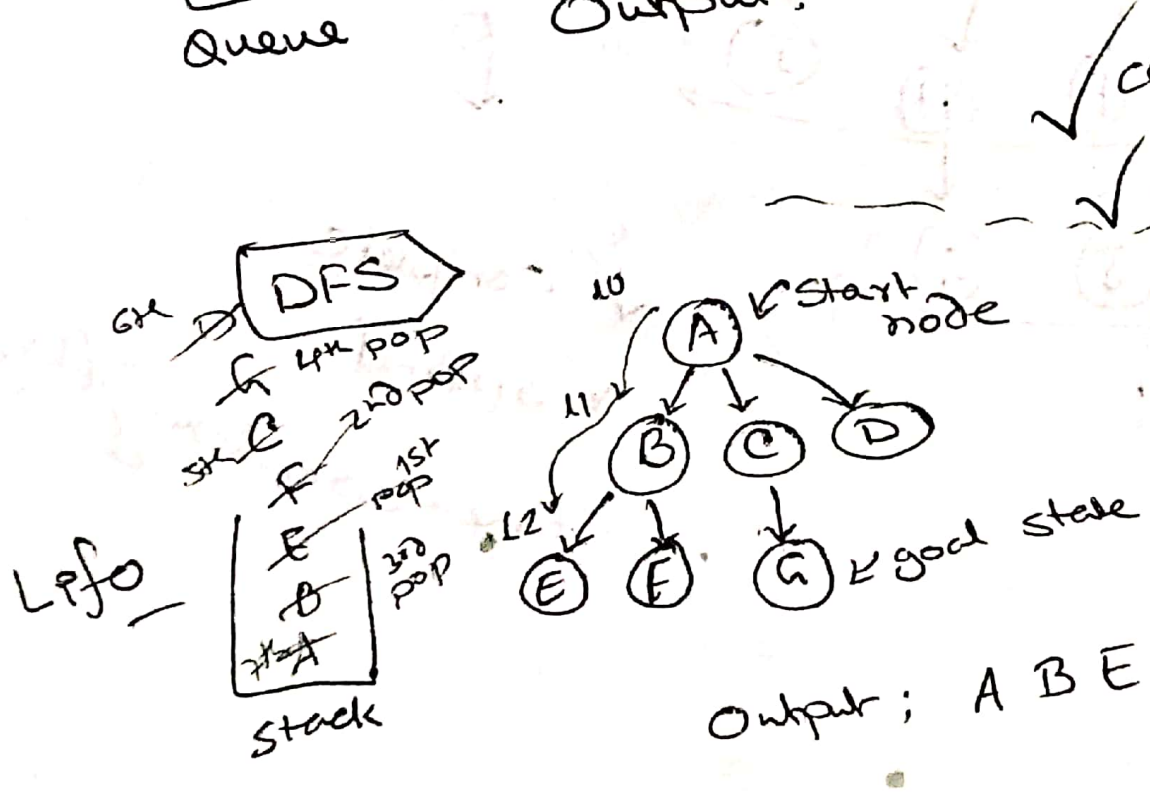
③ Local
(infinite search)



for small graph

Output: A B C D E F G

✓complete (sometimes Guarantee)
✓ *completeness
✓ *optimality

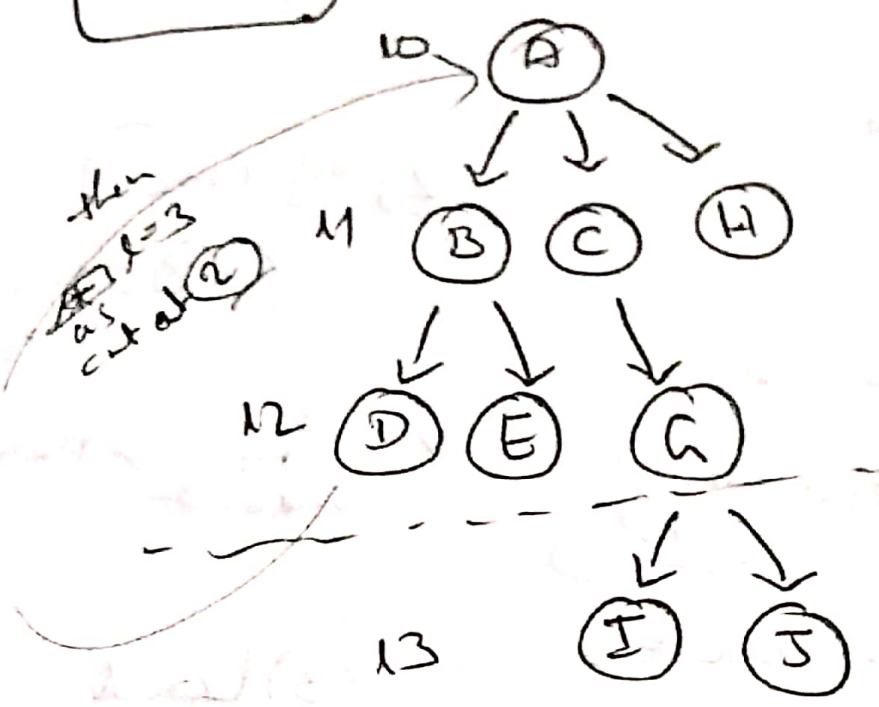


Output; A B E F C G D
terminate

✓ falls in infinite loop if loop present

✓ not complete
✓ not optimal

DLS



limit = 2

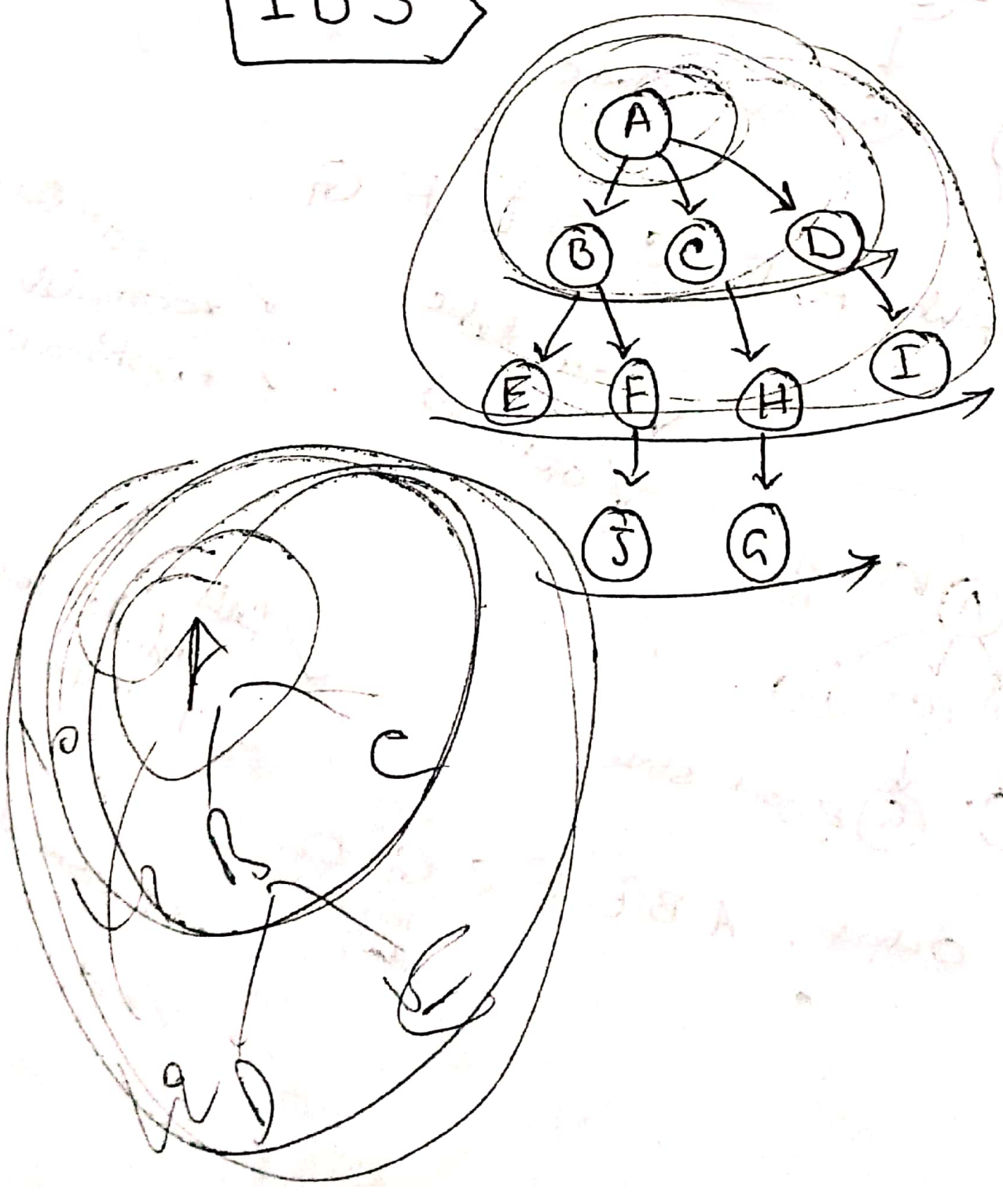
✓ not complete
✓ not optimal

if, goal node out of limit /



if terminate, not getting better goal node /

IDS

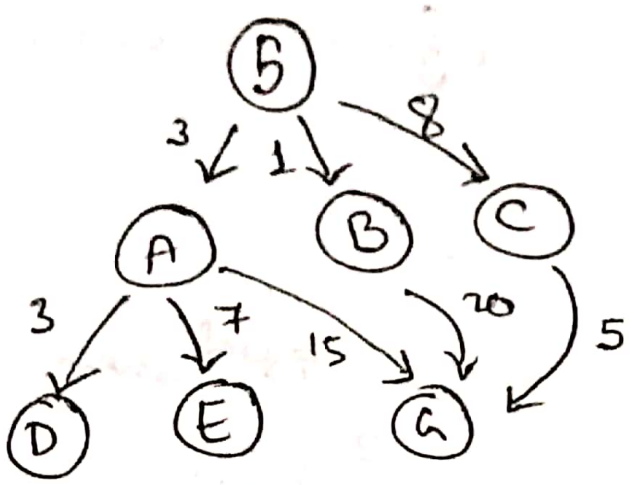


depth limit = 1
depth limit = 1+1
depth limit = 1+1+1

✓ complete
✓ optimal
time-complexity (high)

UCS

cost associated



priority queue

S° | B¹A³C⁸

B¹ | A³ C⁸ G²¹

A³ | 1. D⁶ C⁸ E¹⁰ G¹⁸

(13)

D⁶ | C⁸ E¹⁰ G¹⁸

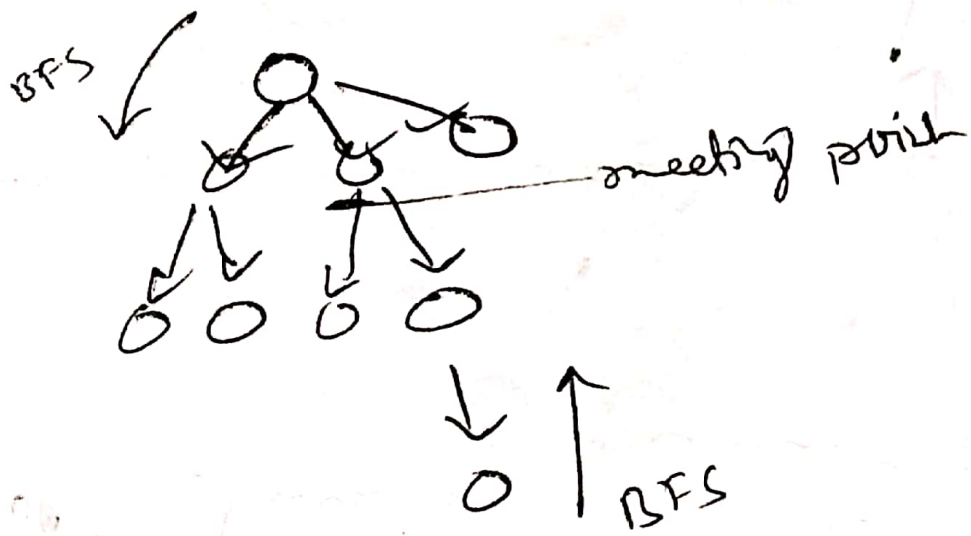
C⁸ | E¹⁰ G¹³

E¹⁰ | G¹³

G¹³ | [ ]

(SOLVED)

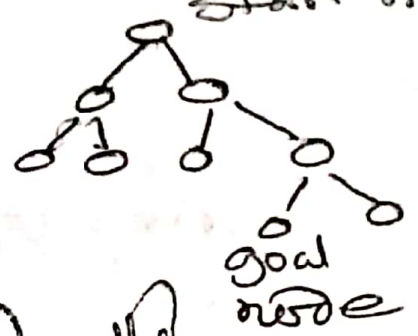
BDS



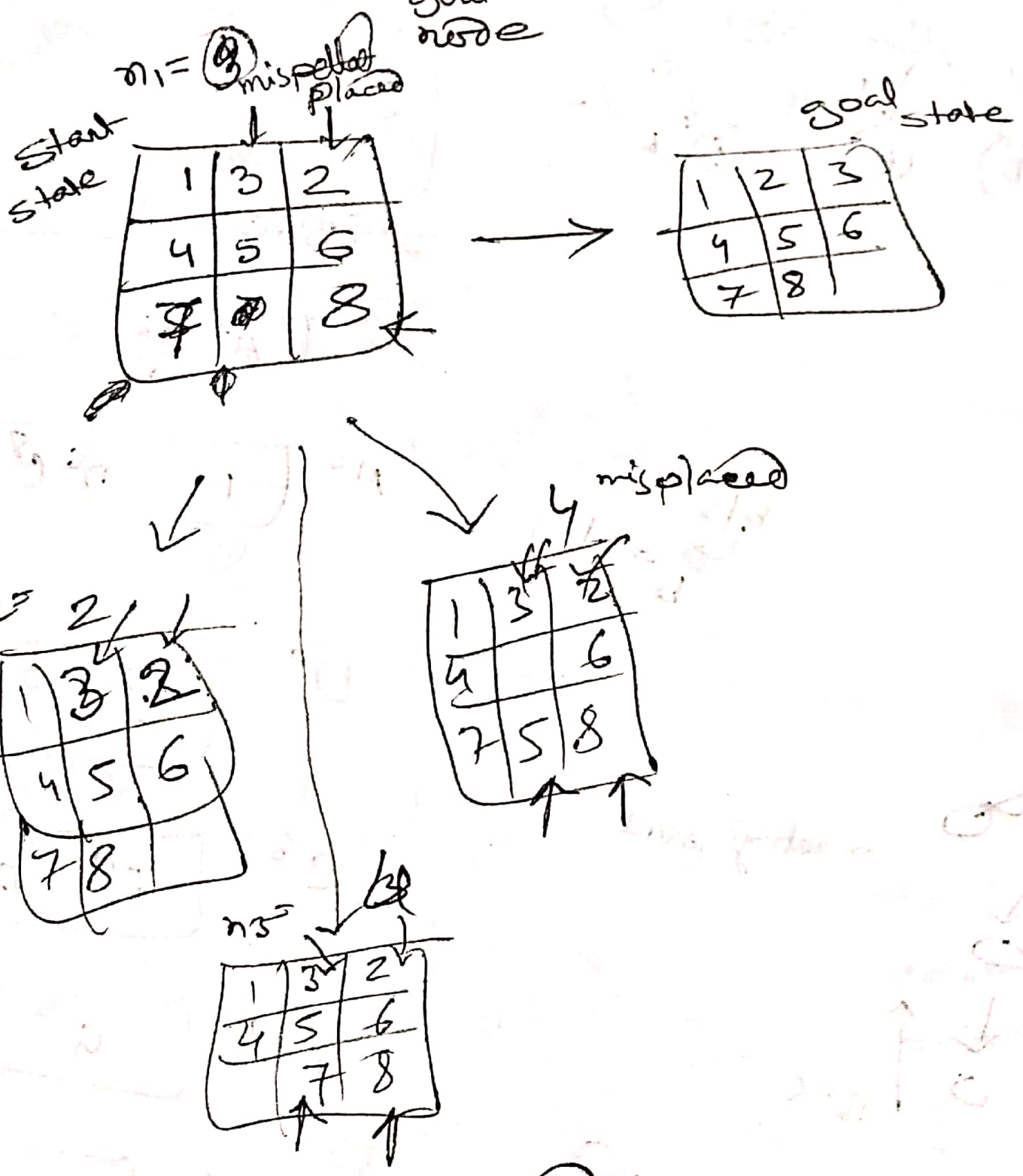
bfs → b^{d+1}

time-complexity (log) : $b^{d/2}$

State - space graph (all possible combined)



(finite structure)



approximation forward → informed search

heuristic, $h(n_1) = 3$

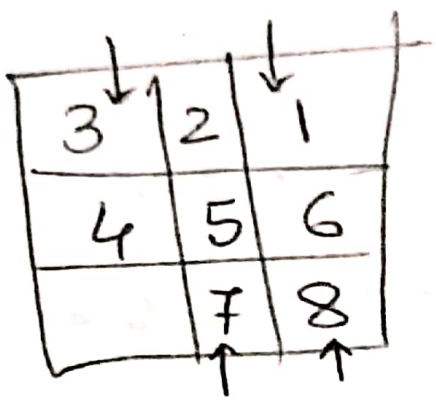
$h(n_2) = 2$

$h(n_3) = 4$

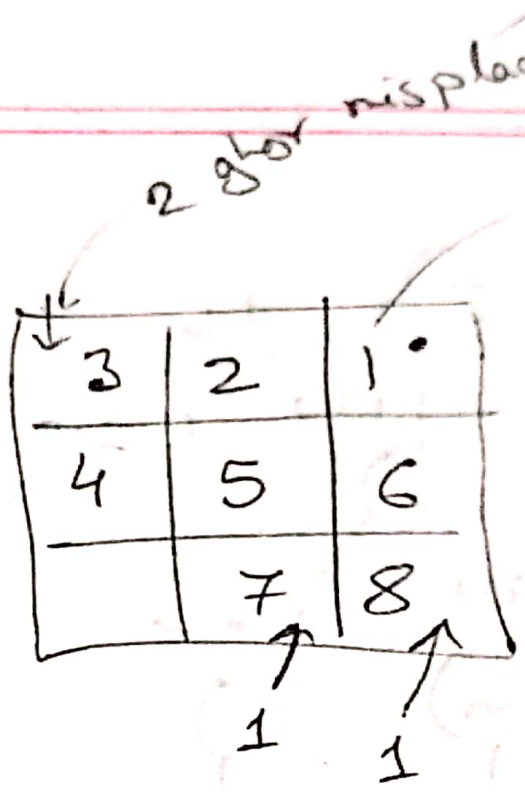
+ve no. -ve no.

less → high

if we want to.



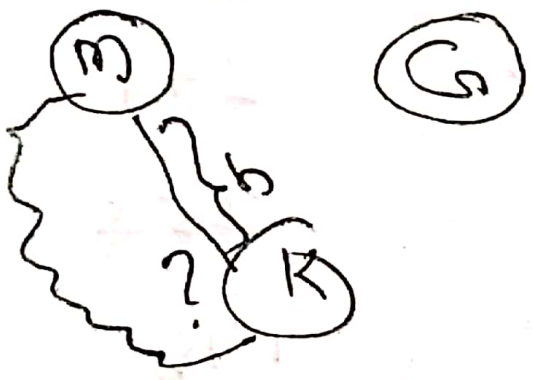
4 misplaced



2+2+1+1
↳ = 6

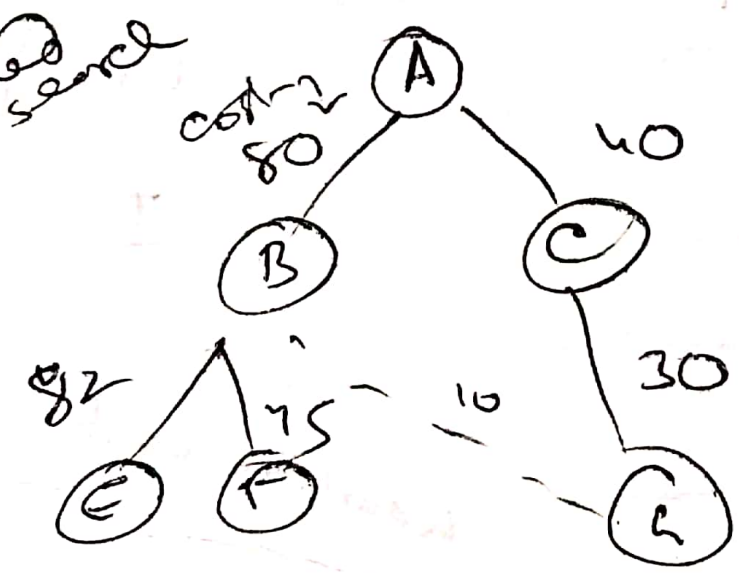
- max heuristic displaced

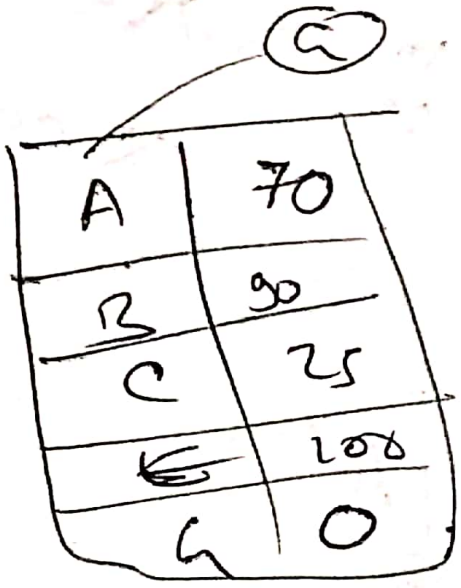
- more accurate.



heuristic better
↓
better search

informed search



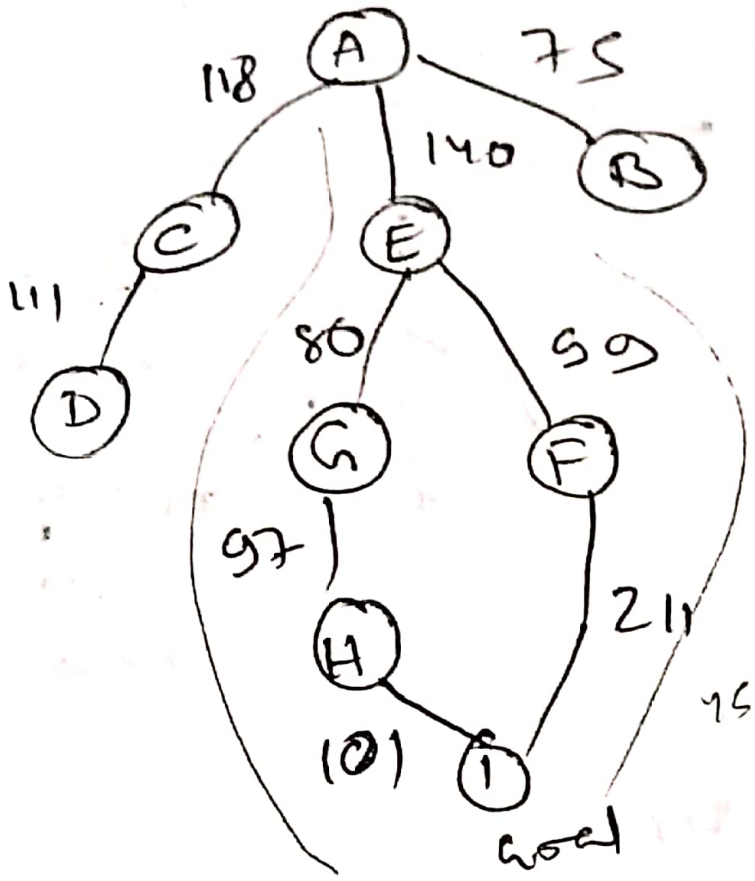


heuristic table

✓ Greedy best first (heuristic table)

✓ A* search (cost value)

path
A → E → F → I
140 + 99 + 211
= 450



State	heuristic; h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

In general → ✗ optimality NO
✗ complexity NO

h(n)
GBF

A 366
start node

heuristic cost.

A 366 expand
C 329 E 253 B 374
(lowest)

E 253
C 329 B 374 G 193 F 178

F 178
C 329 B 374 G 193 I 0

I 0
C 329 B 374 G 193

A* search

for A,

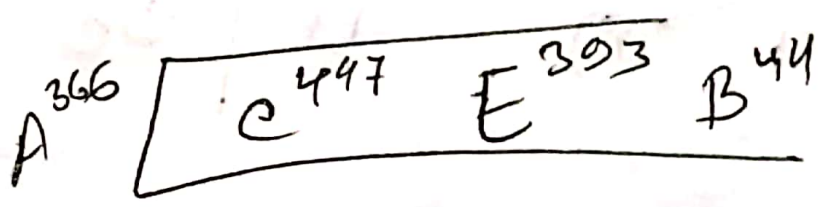
366 + 0

C,

E,

140+253





approach

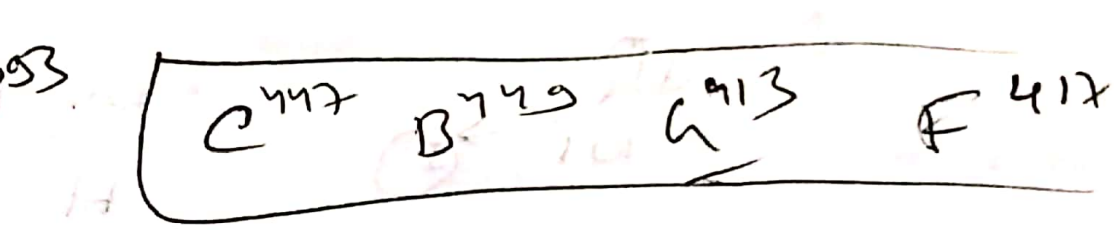
393 - 253

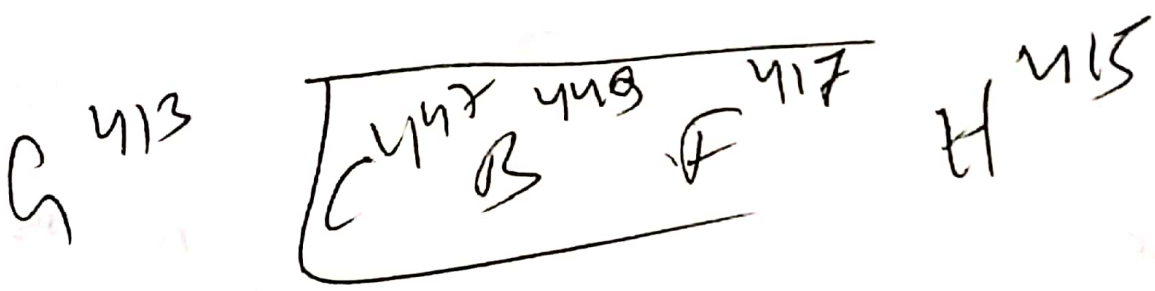
= 140 + 60 + 193

(~) & E.

140 + 0.9 x 178

= 417



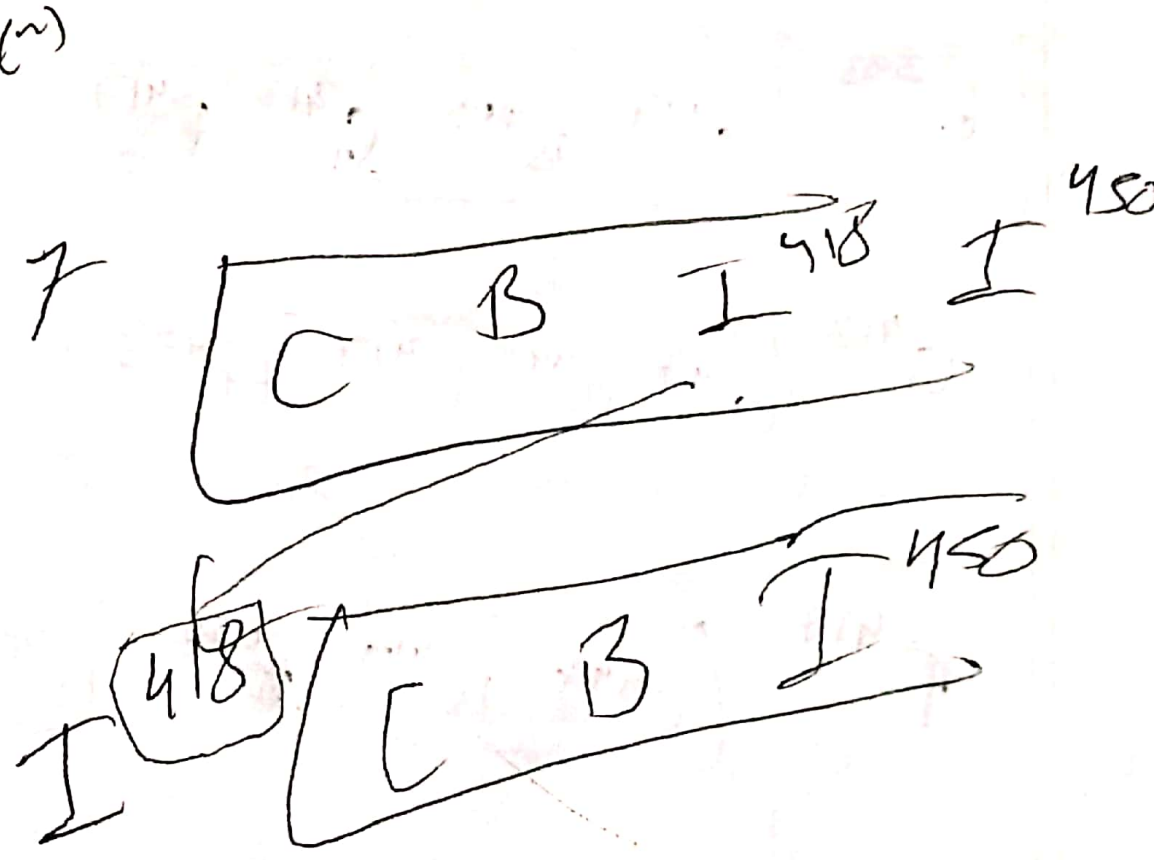




415 - 398

= 317 + 101 + 17

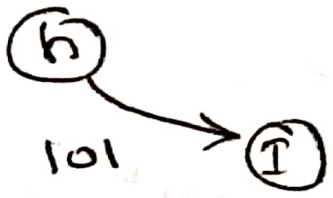
= 418



C-7

admissible heuristic

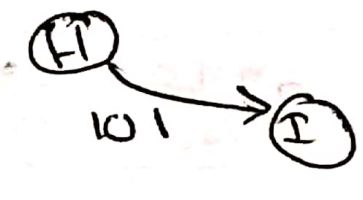
↳ from actual path cost; h(n) should be low.



State	h(n)
H	98

(admissible as less than actual path)

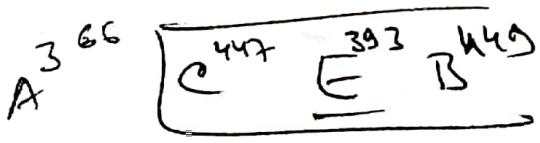
A* not optimal

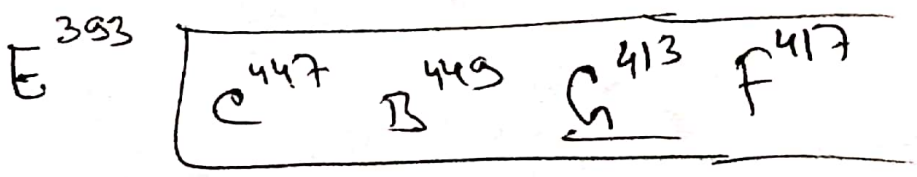


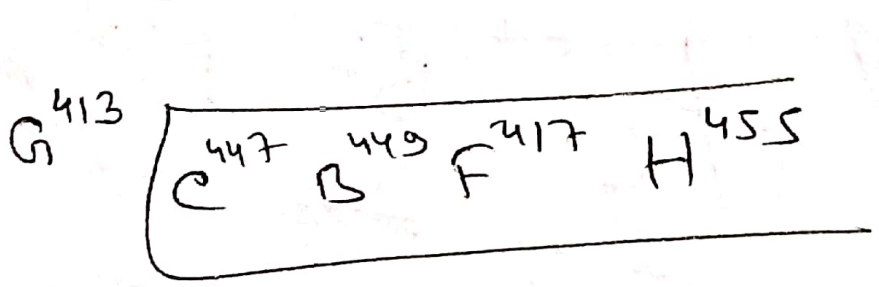
State	h(n)
H	138

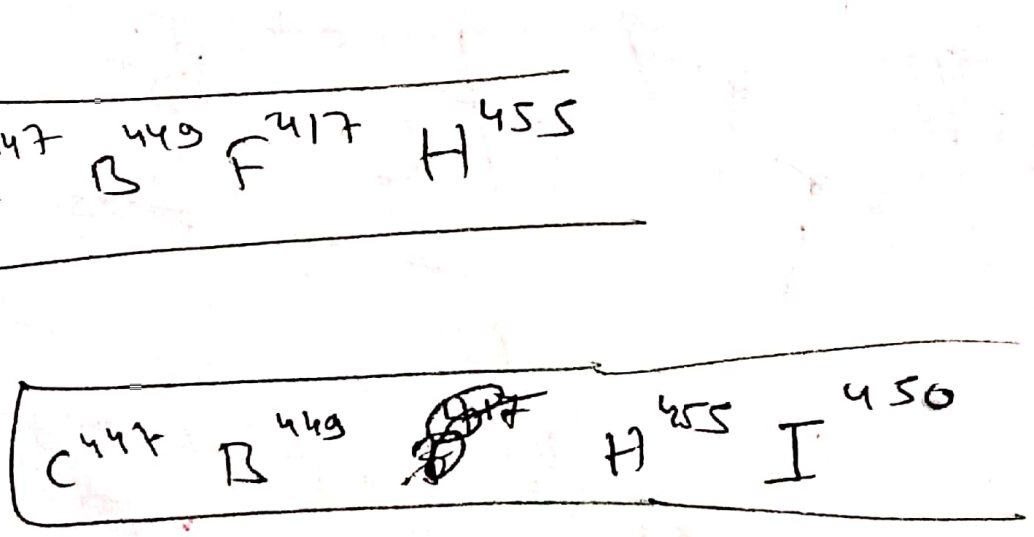
A* optimal

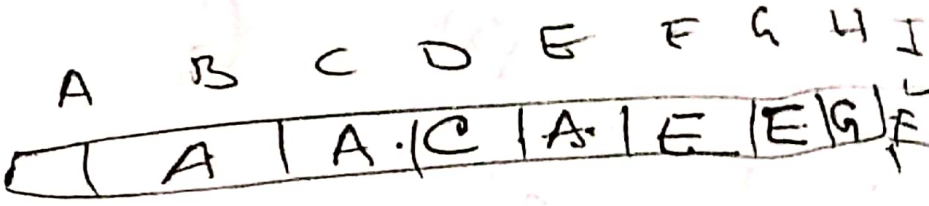








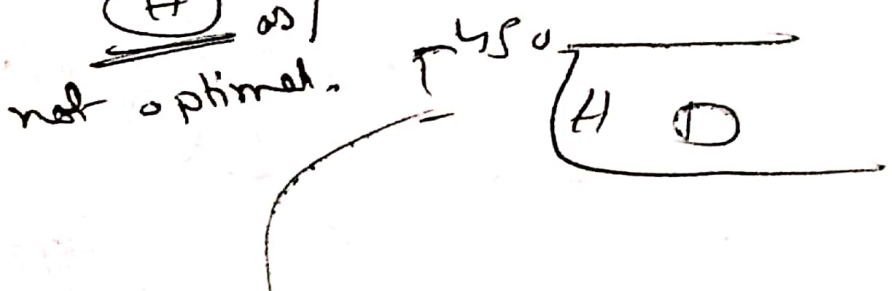




Path = A → E → F → I

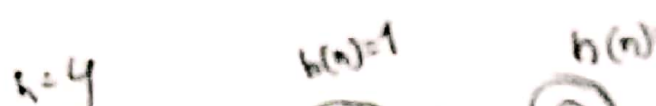
as not admissible

H as ↑ not optimal.



B 449

Consistency :-



	h(n)	c(n)
G	0	2
		= 2

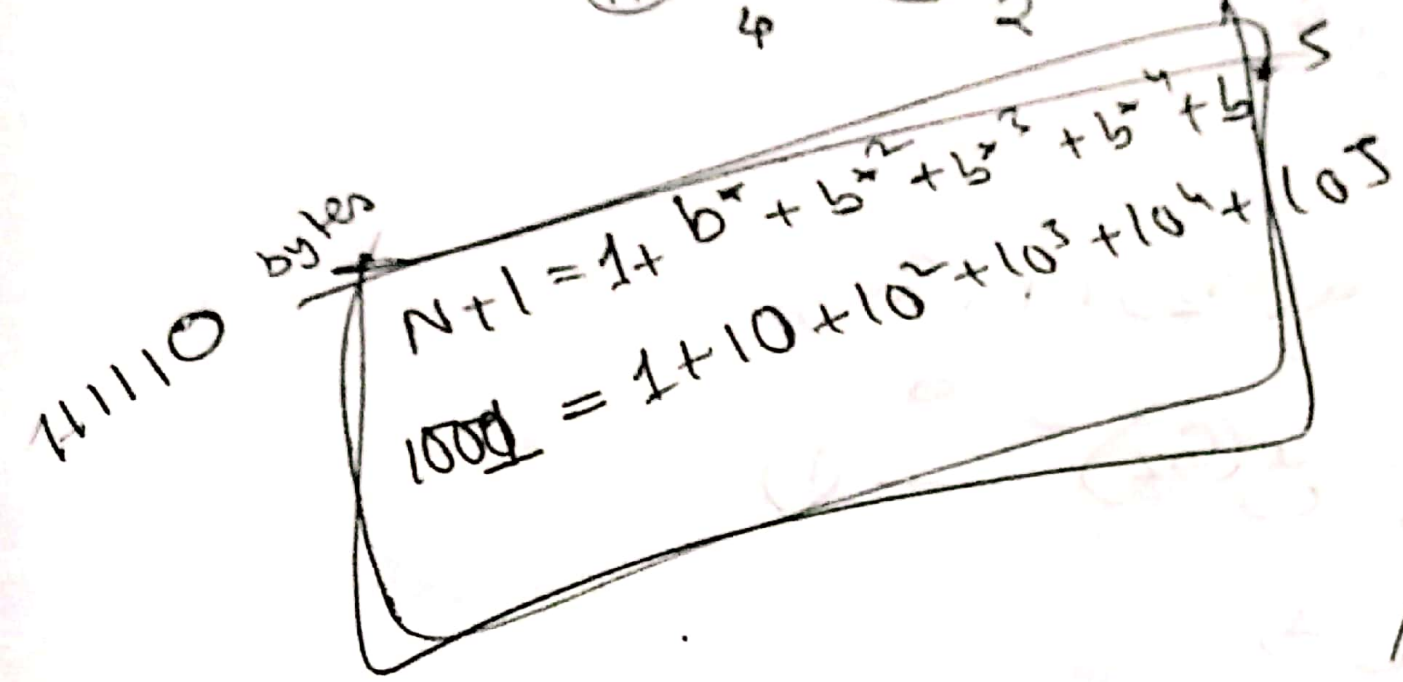
Then $h(B) \leq 2$

B | 1 + 4

= 5

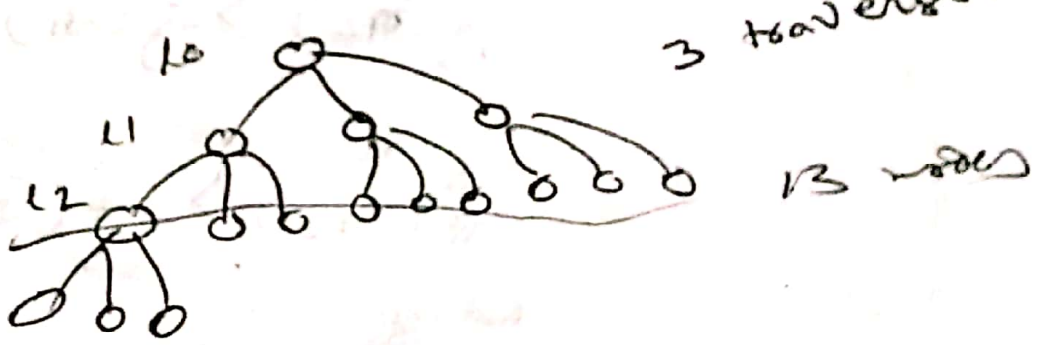
Then $h(A) \leq 5$

// maintains consistency.



A* → branching factor,

Suppose, ~~avg~~ b.f → 3,



eg:

$b^* = 2.5$

$d = 2$

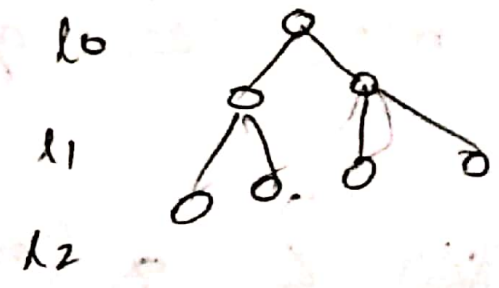
$N = ?$

$N = (1 + b^* + b^{*2}) - 1$

$= (1 + 2.5 + 2.5^2) - 1$

$= 1 \times 5^2 \cdot b^{*2}$

b.f → 2, depth 2,



~~avg~~ b.f put to less-

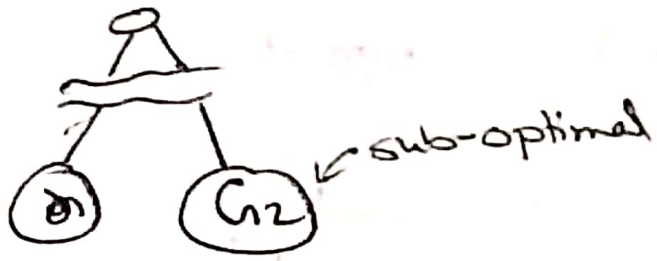
formula : $N + 1 = 1 + b^* + b^{*2} + b^{*3} + \dots + b^{*d}$

N = visited nodes,

optimal ✓

A*

admissibility proof

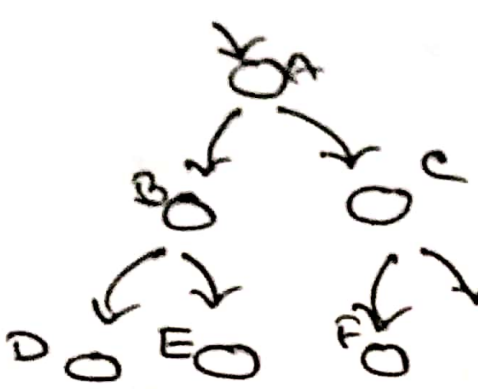


(G)

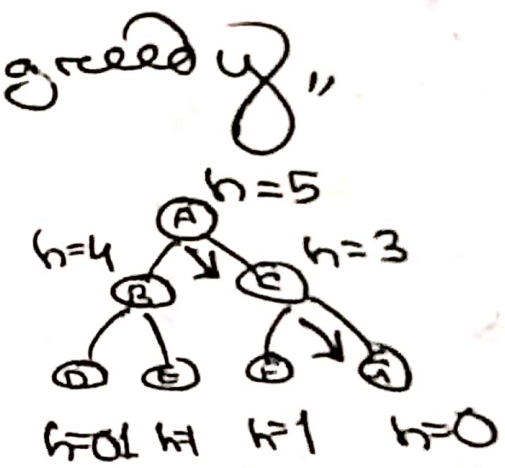
$$f(G_2) = g(G_2) + \underline{h(G_2)}$$
$$\underline{\quad\quad\quad

Local search —

① Time complexity.
② Space complexity. — more concerning; long time run;
gigantic infinite state space;



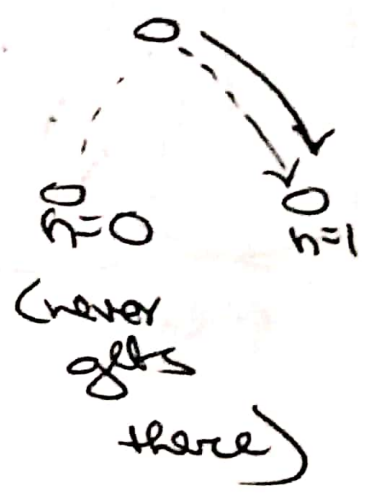
greedy"



A
A B C
C B F G

"we will not do back-tracing eg. blind folded game -

error stays ↳ only take short;
forget others;
lost ;;

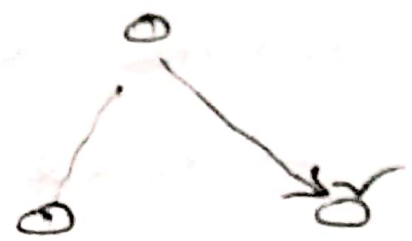


we're happy
↳ sub optimal goal / soln
↳ "Local search"

Local search's variant

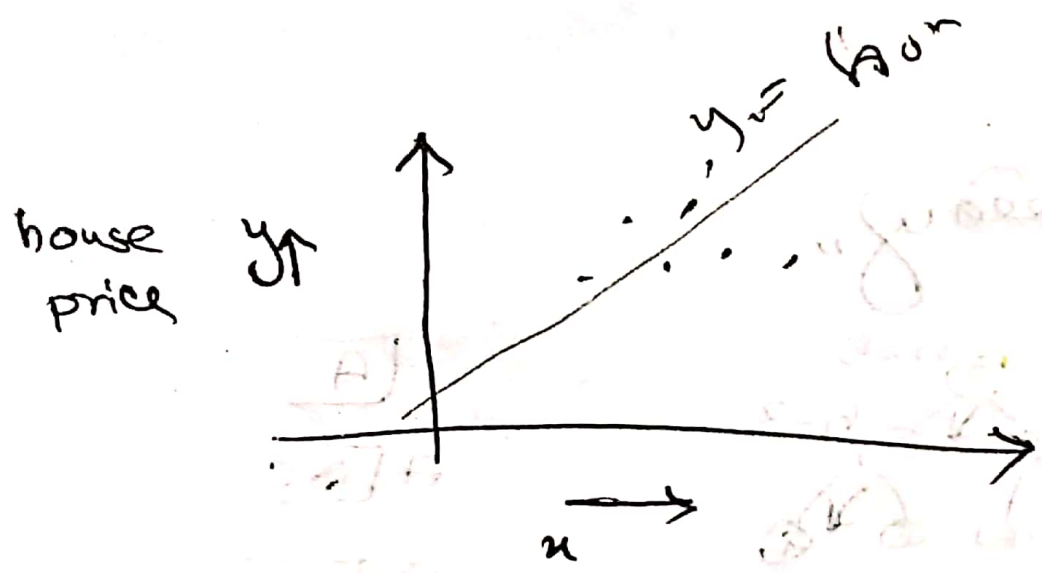
* hill-climbing (works on greedy, low cost)

{not need of perfect goal;}

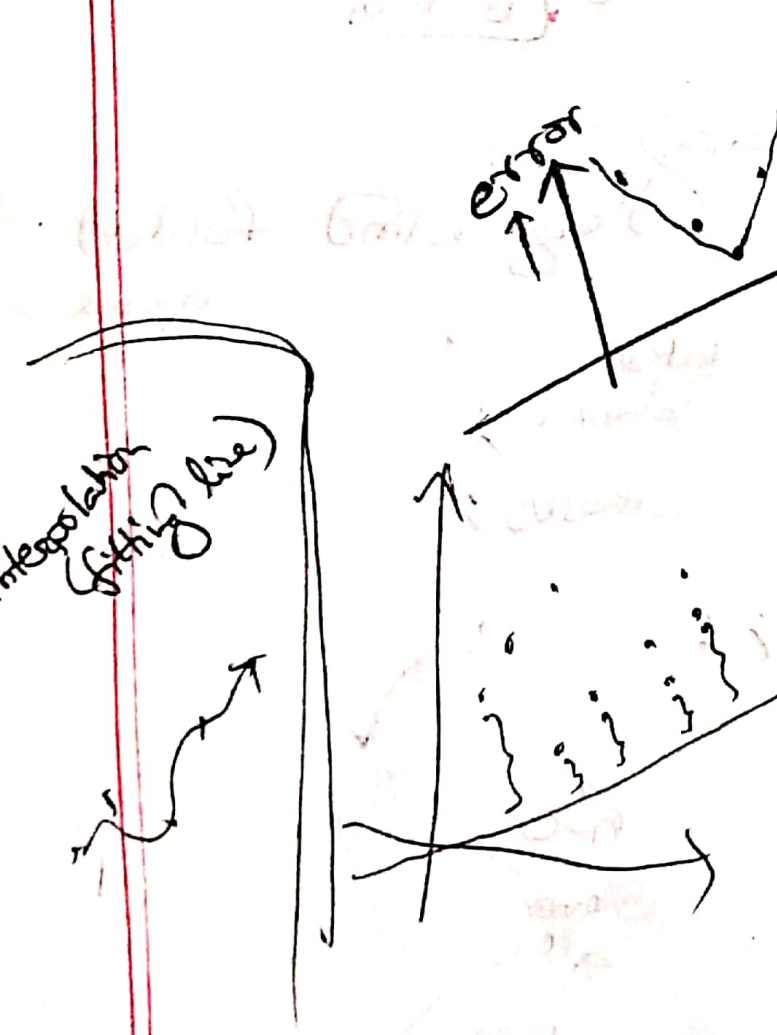


finite state space (नहीं)

↳



(to find outside of pts, then →) "predict it"



$h_0 x + h_1$ → (fitting line)



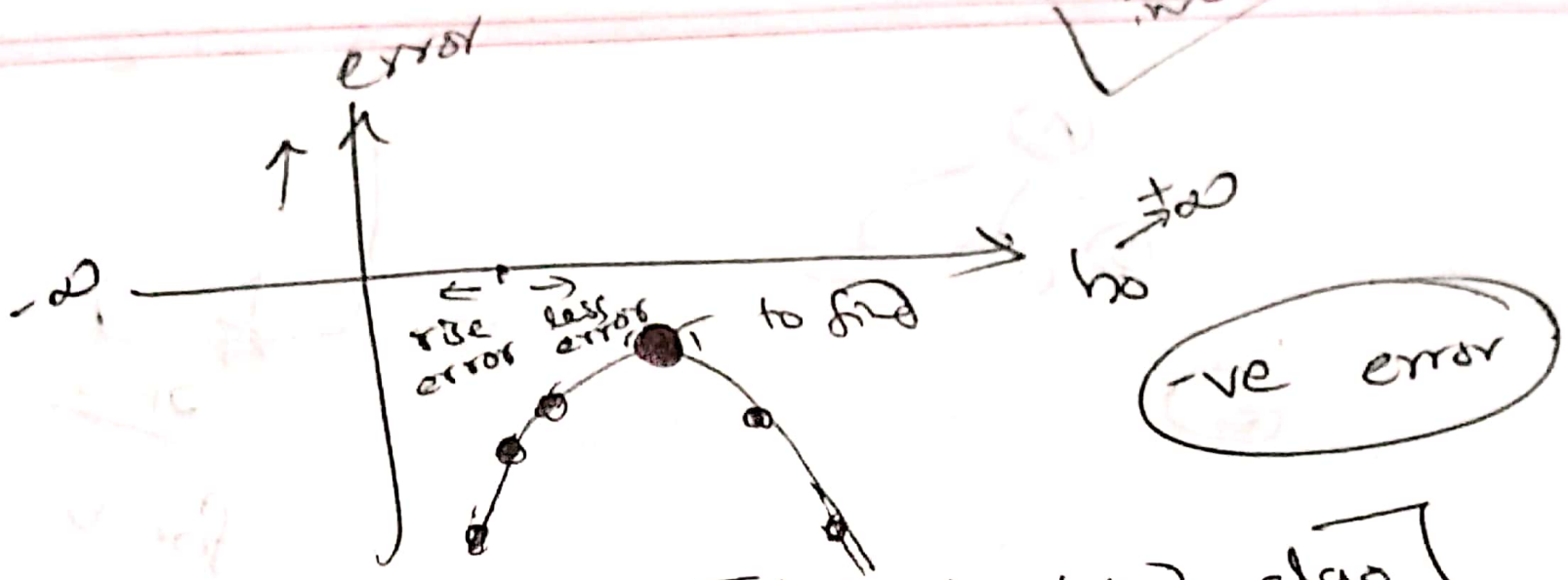
error दिए जाते

$y = h_0 x$

randomly value of $h_0 x$

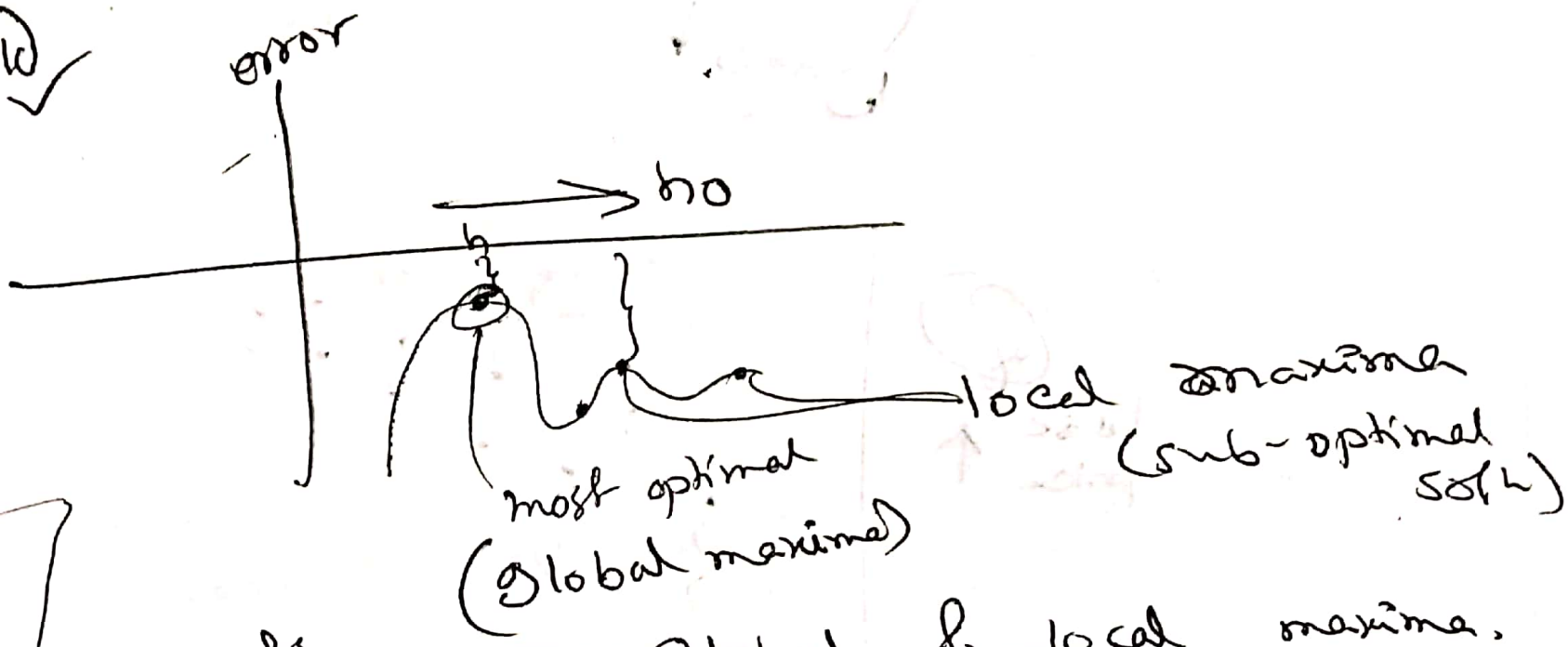
to solve equation

infinite search space



✓ [hill climbing algo]

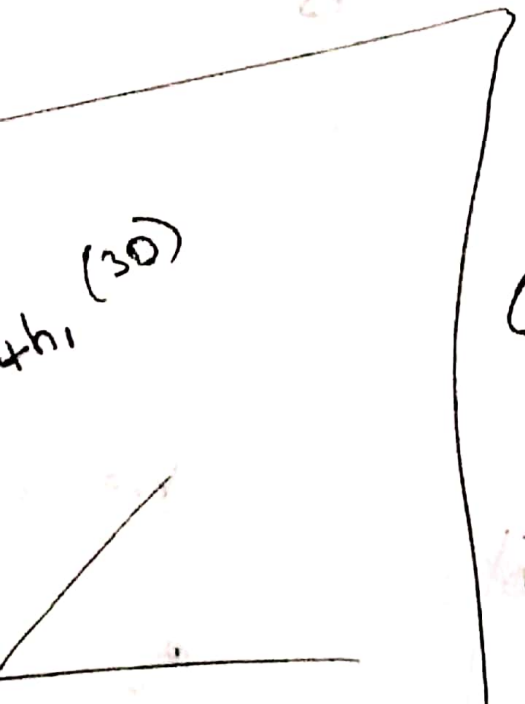
real-world ✓

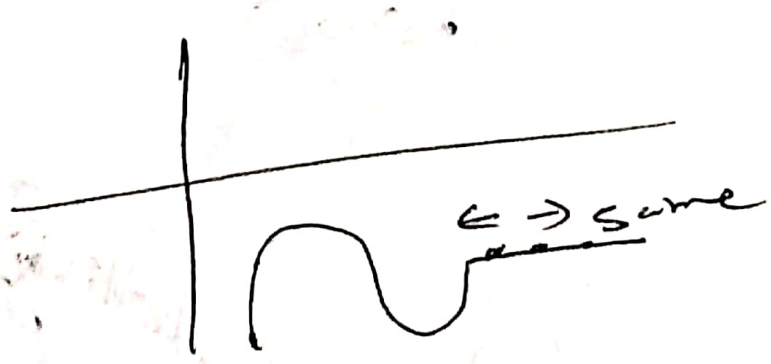


h_0
2D

(3D)

① probs → stuck in global & local maxima.

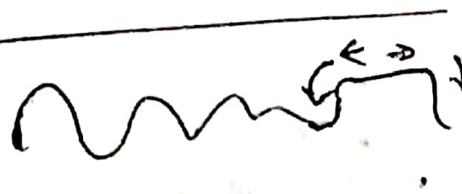




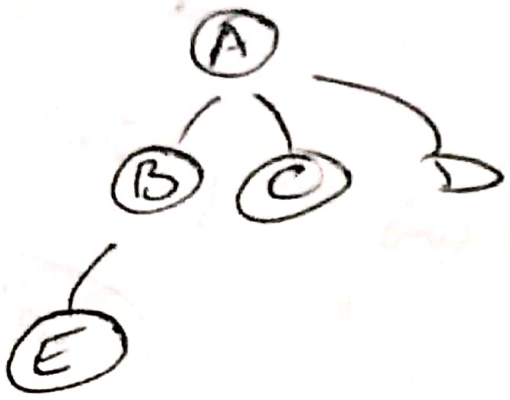
② ll plateaus
(flat line)



~~global~~ [local maxima with flat plateau]



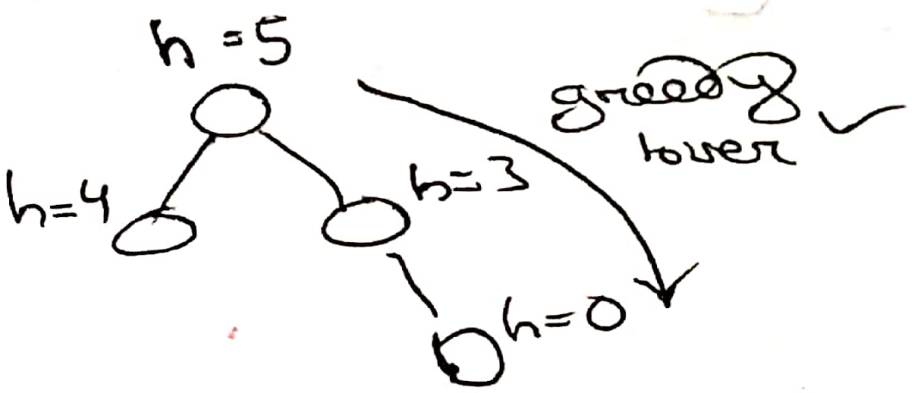
③ ~~Rig~~ (Rid yes)
-ve -ve





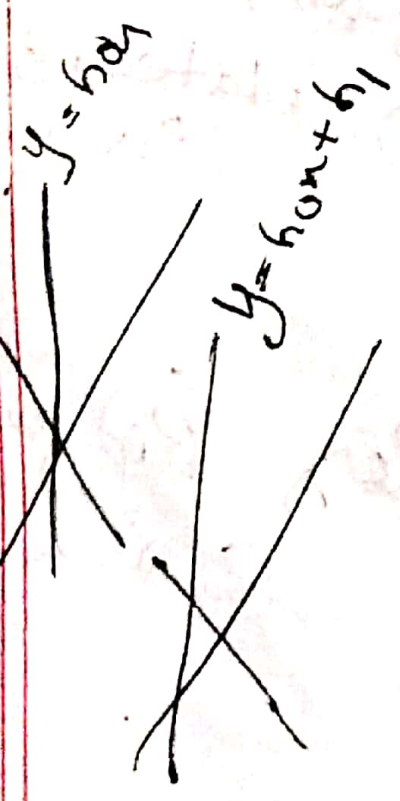
DFS

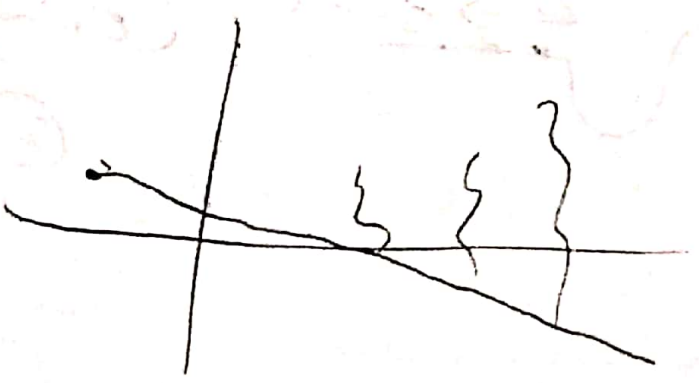
for back tracking uses tree ✓

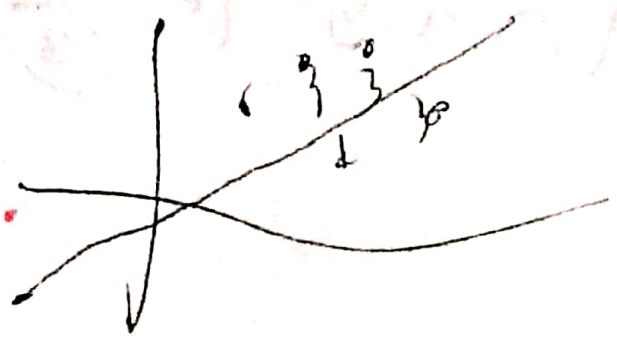


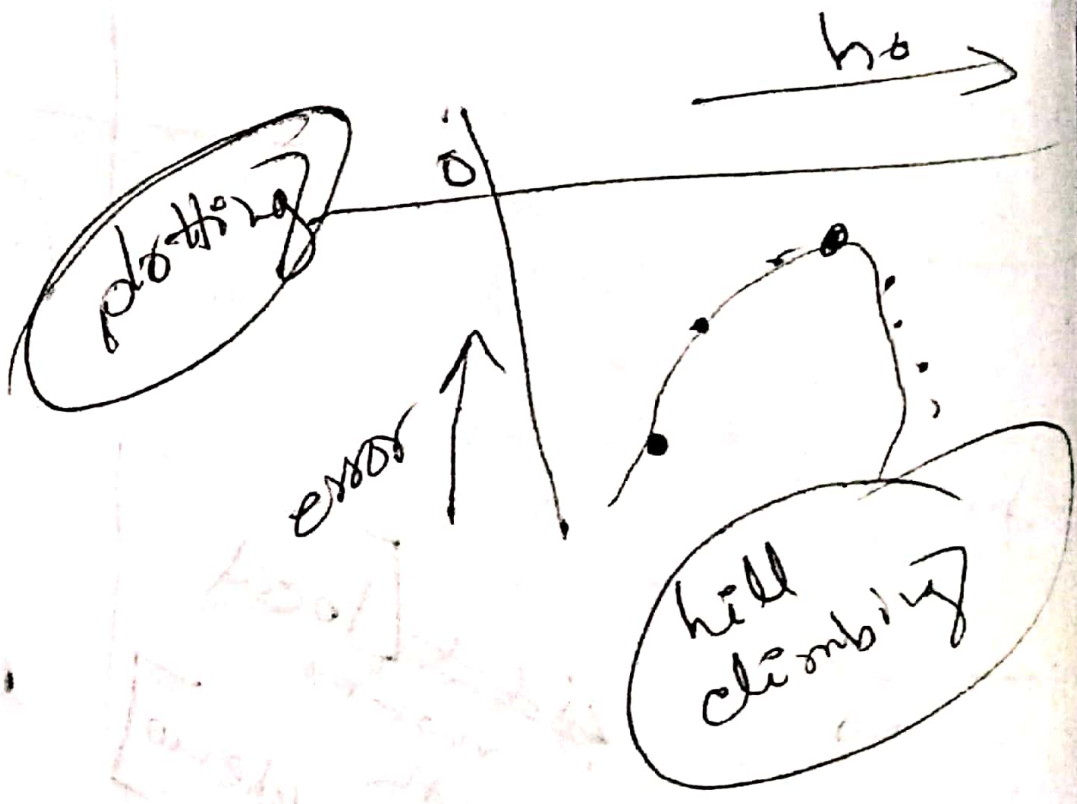


$y = h_0 x$









Rest

* check/compare better values

Randomize

Random start

problem Reformulation

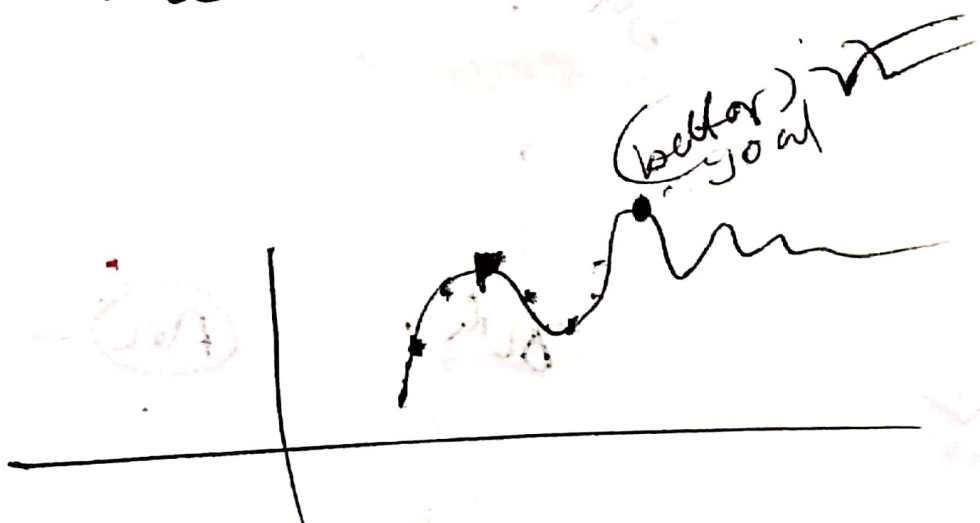
Reformulation

width	length	price

→

area	price

Simulated Annealing,



error expectancy ↓ as we proceeds.

$C = C_{init}$

for $T = T_{max}$ to T_{min}

$E_C = E(C)$

$N = Next(C)$

$E_N = E(N)$

$\Delta E = E_N - E_C$

// Ec Energy

// Next state : N

if ($\Delta E > 0$)

$C = N$

else if

($e^{\Delta E/T} > rand(0,1)$

Genetic algo ⟹

- population
- fitness function $\Sigma x_i \cdot P(x_i)$ (ch-1)
- crossover
- mutation (random included)

A_1 A_2 ~~A_3~~ A_4

next gen: A_5 A_6 A_7 A_8 better

3rd gen: A_9 A_{10} A_{11} A_{12} better

:

20th: A_n - goal

sample — random

02113 02212

02212 02113

03212 x ——

21230 ——

1st gen

2nd gen

Prob of Backtracking Search,

	1	2	3
	-	-	-
	-	-	□

In Sudo code, the 3,3 cell has no possib value.

But, backtracking search we doesn't detect this until all variables of a row/ column -- are assigned.

*Improving Backtracking

Ordering (which variable should be assigned next)

(which order should go)

filtering (can we detect inevitable failure early)

Structure (can we exploit the prob structure)

4 Queens using Backtracking + Forward Checking.

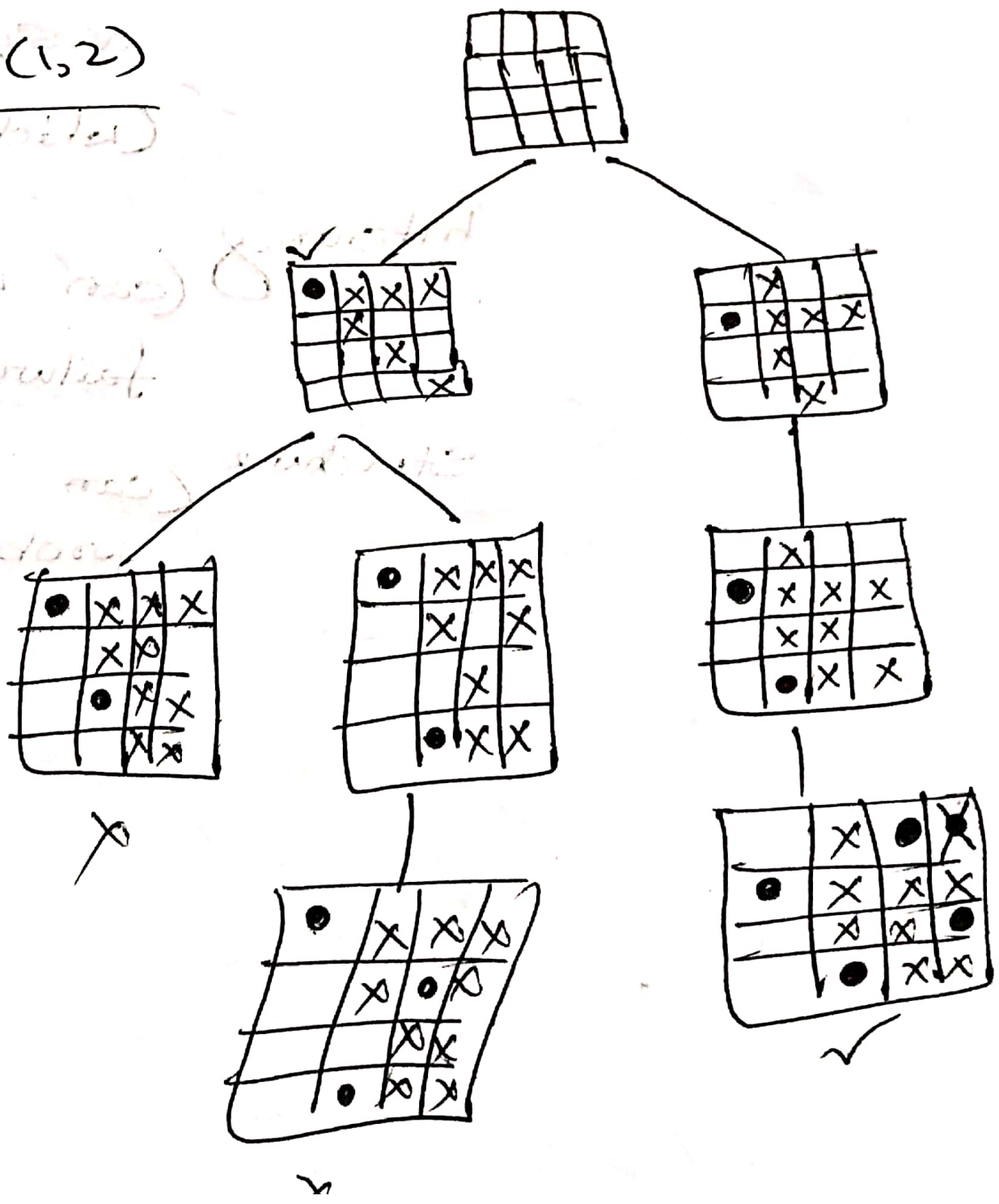
✓ Filtering → keep track of domains for unassigned variables and cross off bad options.

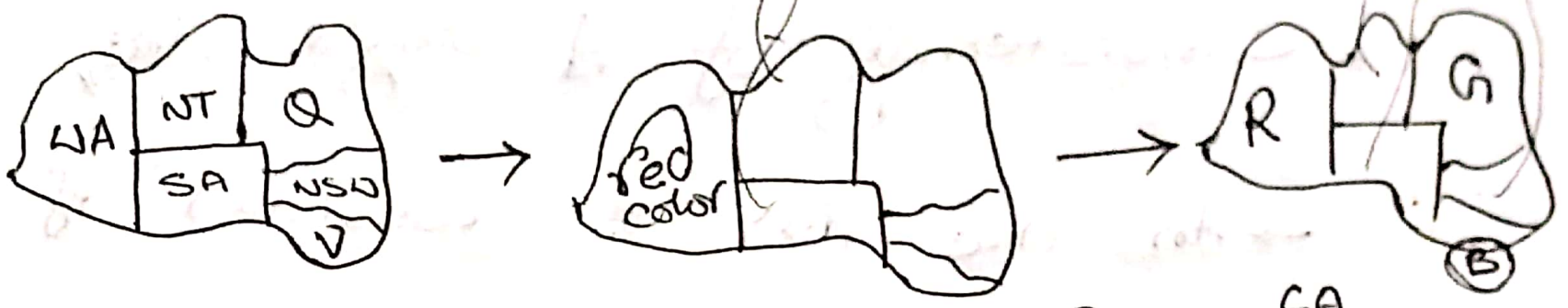
✓ Forward checking → cross off values that violate a constraint when added to existing assign

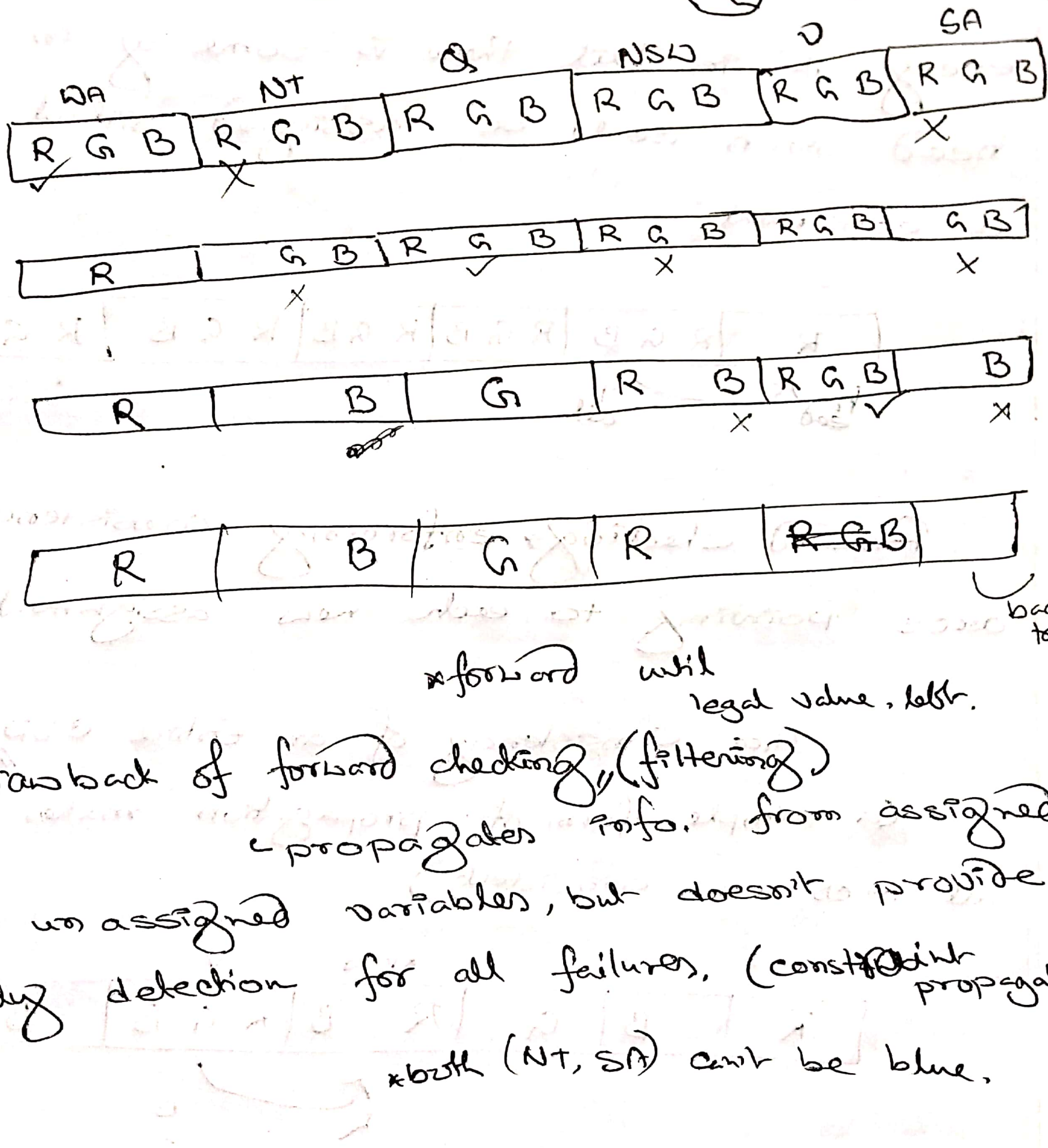
Domain of $Q_1 = (1, 2, 3, 4)$

$Q_2 = (\cancel{1}, \cancel{2}, 3, 4)$

(1,1), (1,2)







*forward until legal value, both.

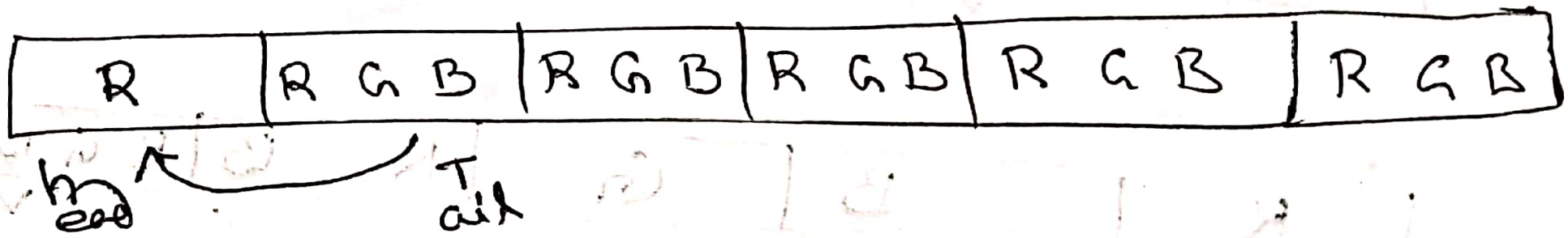
Drawback of forward checking (filtering)

↳ propagates info. from assigned to unassigned variables, but doesn't provide early detection for all failures. (constraint propagation)

*both (NT, SA) can't be blue.

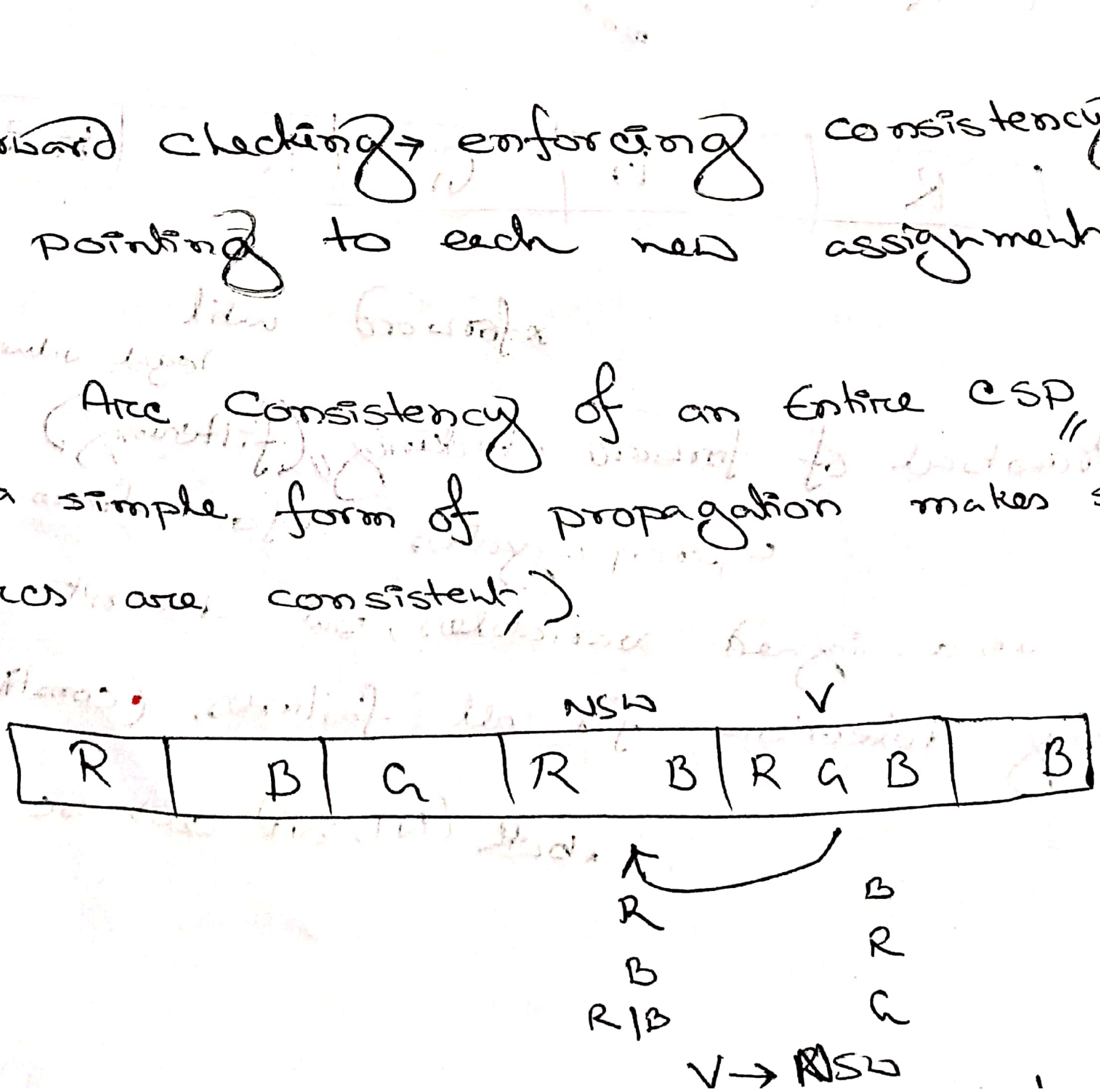
* Consistency of A Single Arc,

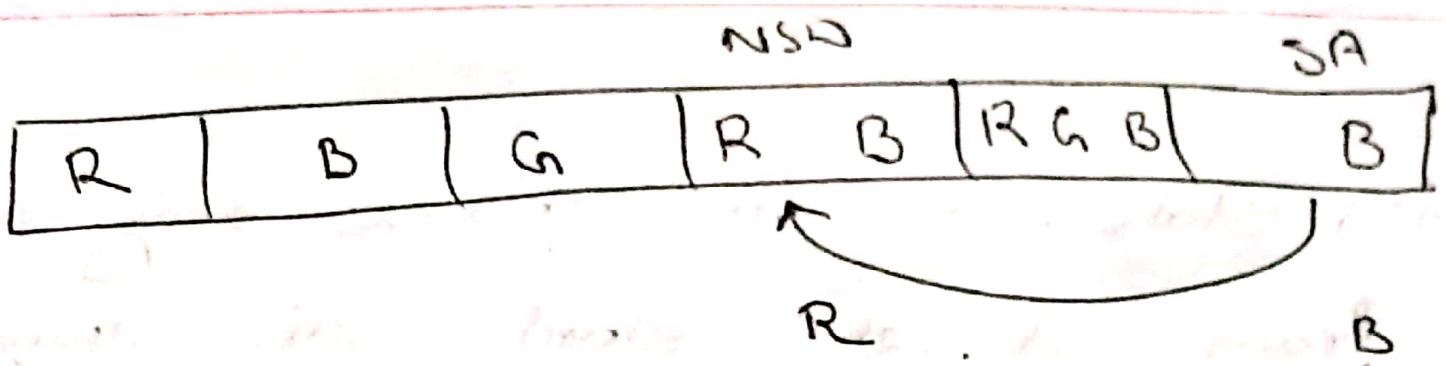
→ An Arc X→Y is consistant if for every x in tail there is some y in the head which could be assigned without violating a constraint,



Forward checking → enforcing consistency of arcs pointing to each new assignment,

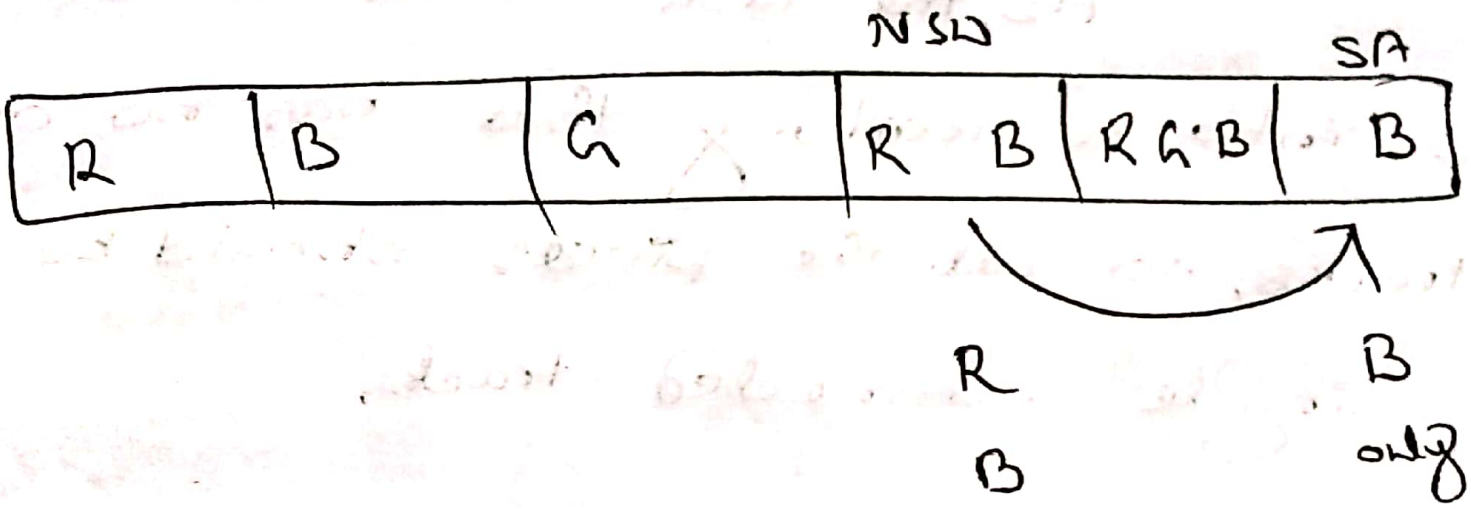
Arc Consistency of an Entire CSP,
(a simple form of propagation makes sure all arcs are consistent)





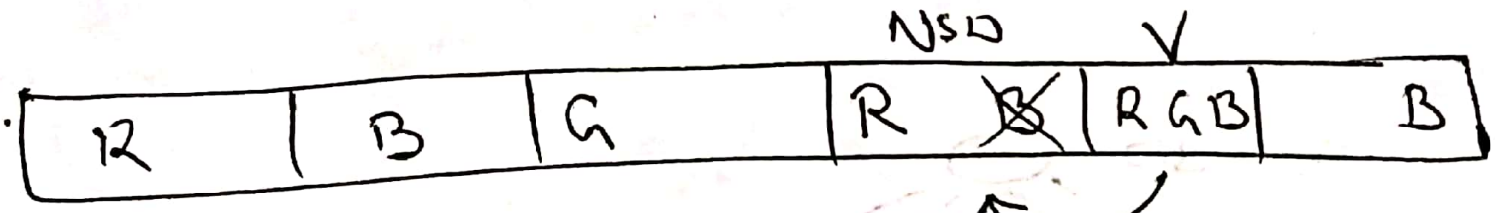
So consistent.

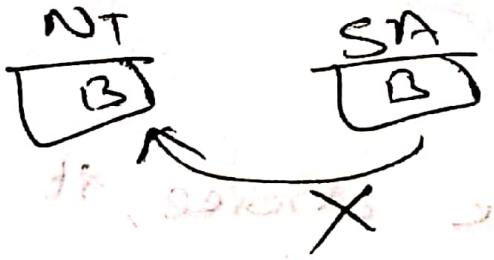
SA → NSD



So, not consistent.

NSD → SA (X)





(inconsistent) now

*note,

- If x loses a value, neighbors of x need to be rechecked.
- Arc consistency detects failure earlier than forward checking.
- Can be run as a preprocessor or after each assignment (called Maintaining Arc Consistency/MAC)

Arc

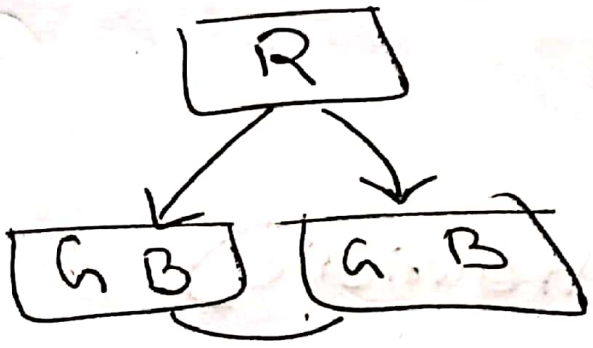
Enforcing Arc Consistency in a CSP.

* Runtime: $O(nrd^3)$

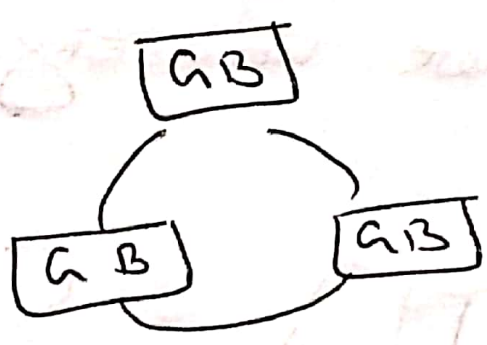
Limitations of Arc consistency.

(after enforcing arc →)

- can have one solution left.
- can have multiple solutions left.
- can have no solutions left-
  (and not know it)



✓



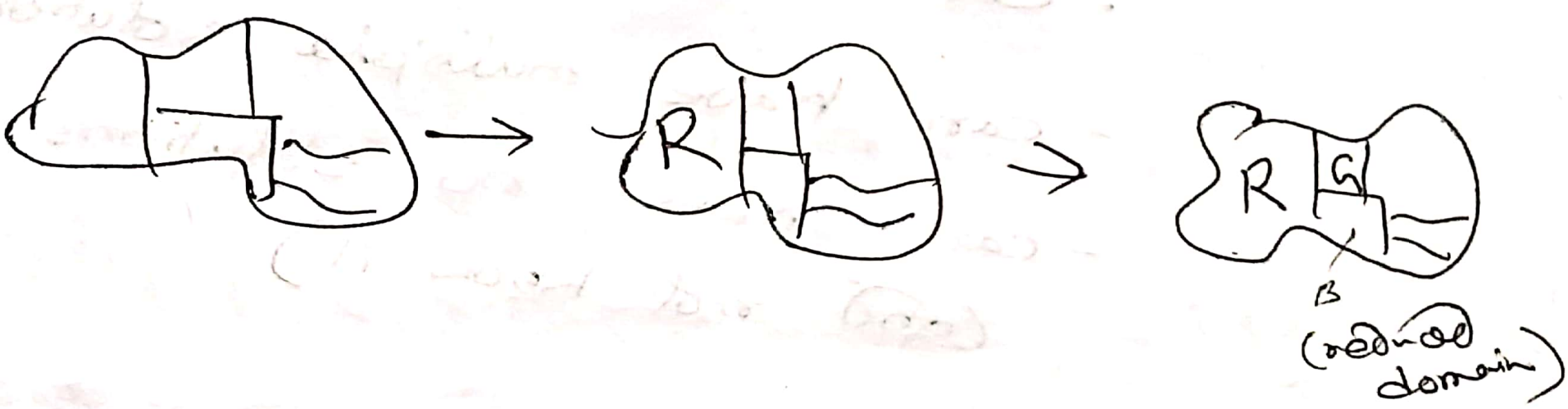
failure.

* Ordering

(minimum remaining values)

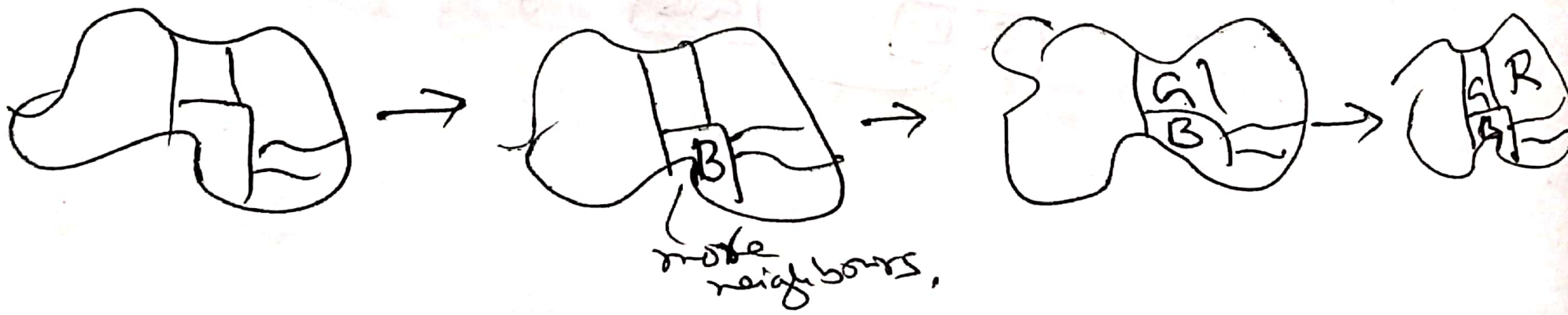
MRV

✓choose the variable with the fewest le
values in its domain,



(degree heuristic)

✓choose variable with largest number of constraints on other unassigned variables,



✓attempts to reduce the branching factor on future choices,

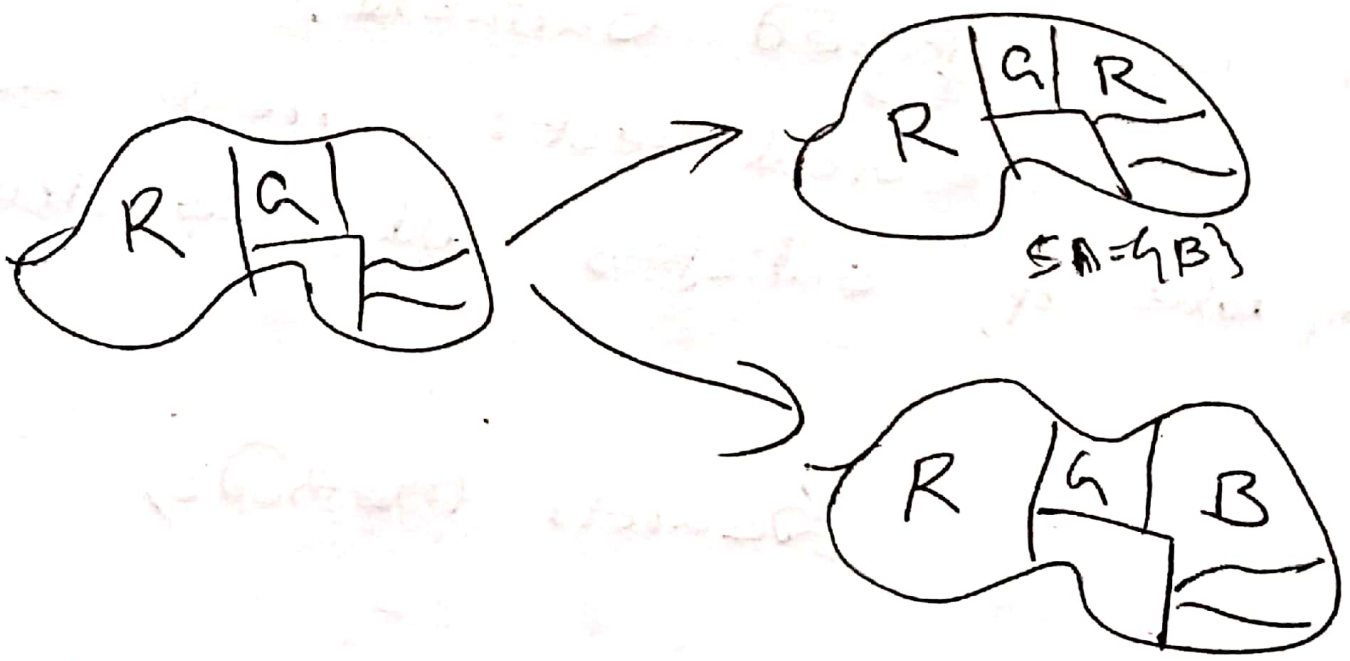
✓useful as tie-breaker,

(least constraining value)

* the one that rules out the fewest values in the remaining variables.

* It may take some computation to determine this (eg. retaining filtering)

* leaves max. flexibility for subsequent variable assignment



Solving CSP,

standard search Formulation,

-states defined by values assigned,

-Initial state: empty assignment, {}

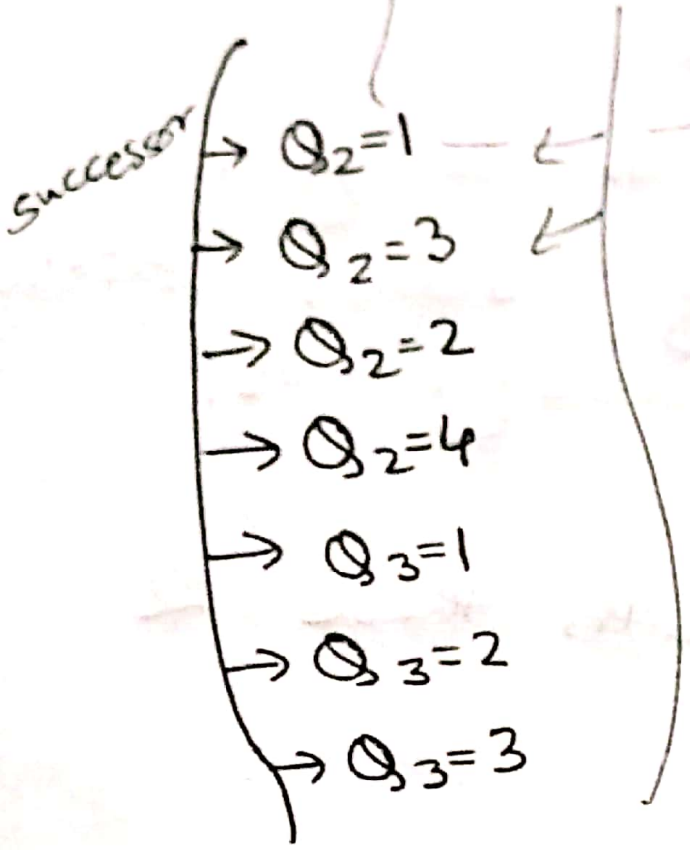
-Successor function: assign a value to an unassigned variable.

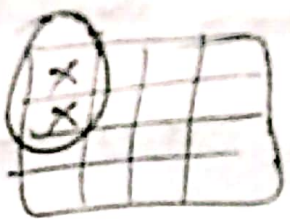
-Goal test: current assignment is complete & satisfies all constraints.

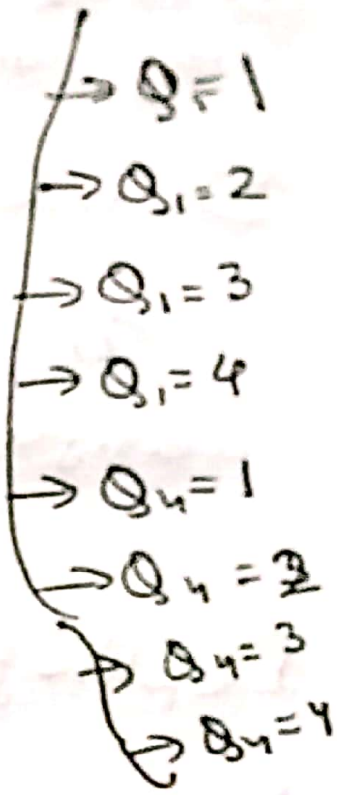
Search methods,

→ { }

→ $Q_{s_1}=1$ | $Q_{s_1}=2$ | $Q_{s_1}=3$ | $Q_{s_1}=4$ | $Q_{s_2}=1$ | $Q_{s_2}=2$ | $Q_{s_2}=3$ .....







$Q_{s_2}=1, Q_{s_1}=1$; $Q_{s_1}=1, Q_{s_2}=1$

IIbfs not good enough

↳ (csp).

✓ Backtracking Search,

① one variable at a time,
eg. WA = G then NT = R

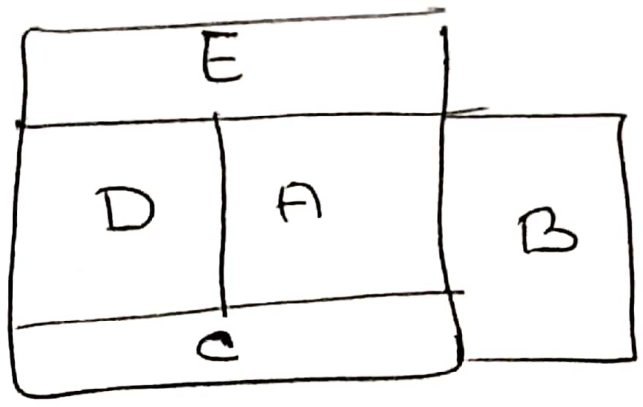
② check constraints as we go,
eg. consider only values which do not conflict previous assignments.

* Depth-first search with these 2 improvements, → Backtracking.

eg. mapping; 4-Queens
(map color) → (sudoku)

Constraint Satisfaction,

- map coloring.



Variables A B C D E

domains {R, G, B}

constraints A ≠ B, B ≠ ...

one solution.

A = R,
B = G,
C = B/...
D = G,
E = B/...

- constraint graph,

